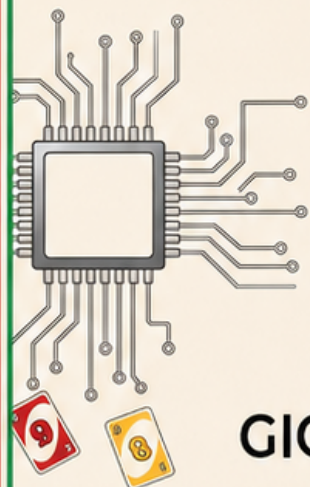




UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO



DOCUMENTAZIONE

caso di studio gioco di carte UNO
di

GIORDANO ALESSANDRO ABBAMONTE
SAMUELE MESSENA
SARA MARIA SBANO



@SGS

Laurea Triennale in Informatica e Tecnologie dell'Informazione
-Esame di *Laboratorio* di Informatica con Doc. G.Vessio
A.A. 2025/2026
Taranto

Indice

Premessa

1) Analisi

- 1.0 Richiesta del committente
- 1.1 Analisi delle specifiche
 - 1.1.0 Carte
 - 1.1.1 Utente
 - 1.1.2 Computer
- 1.2 Requisiti funzionali-tabella
- 1.3 Casi d'uso

2) Progettazione

- 2.0 Modularità: struttura del progetto
 - 2.0.1 Librerie e funzioni
 - 2.0.2 Gestione e registrazione utente
 - 2.0.3 Logica partita e storico partita GAME.H
 - 2.0.4 Componenti software del sistema: variabili, strutture dati e file
- 2.1 Effetti, suoni e grafica
 - 2.1.1 GUI
- 2.2 Gestione Asset
 - 2.2.1 Note di manutenzione
- 2.3 Flowchart gestione gioco
 - 2.3.1 Gestione login e registrazione
 - 2.3.2 Gestione logica di gioco
 - 2.3.3 Gestione archivio e statistiche

2.4 Grafi delle dipendenze

2.5 Flowchart file .c

2.5.1 Main.c

2.5.2 Archivio.c

2.5.3 Audio.c

2.5.4 Game.c

2.5.5 Graphics.c

2.5.6 Menu.c

2.5.7 Menu_scelta.c

3)Codifica

3.0 Stati del MENU

3.1 GAME.C

3.2 MENU.C

3.2.1 Testo ed effetti

3.2.2 Data e ora

3.2.3 Salvataggi e persistenza dati

3.3 MENU_SCELTA.C

3.4 GRAPHICS.C

3.4.1 Pipeline di Redering e Modularità dello Stato

3.4.2 Caricamento Sicuro e Gestione delle risorse

3.4.3 Feedback Visivi e Animazioni UI

3.5 ARCHIVIO.C

3.5.1 Architettura dei Dati e Logica di Impaginazione

3.5.2 Flusso di Lettura del File Binario e Gestione Memoria

3.5.3 Rendering dell'Interfaccia e sottosistema di Navigazione

3.5.4 Sicurezza, Chiusura delle Risorse e Prevenzione Leak

3.6 AUDIO.C

3.6.1 Coerenza dello Streaming e Ottimizzazione del ciclo

3.6.2 Sottosistema degli Effetti e Feedback Acustici

3.6.3 Deallocazione e Chiusura Pulita

4)Testing

4.0 Bug fixing

4.1 Test

4.2 Sviluppi Futuri

Conclusioni

Premessa

Siamo un gruppo di 3 studenti di Informatica dell'Università di Bari e pur essendo ancora alle prime armi abbiamo deciso insieme di affrontare questo progetto e abbracciarne la complessità per sviluppare insieme, le migliori competenze lavorative attraverso le qualità di ognuno di noi. Il nostro logo nasce nell'aprile del 2026, 3 foches, come l'alter-ego goffo di ognuno di noi programmatori alle prime armi, ma c'è una curiosità di fondo: recenti studi hanno portato gli scienziati a scoprire come questi mammiferi tanto apparentemente goffi quanto capaci, soprattutto se *addestrati*, e come essi riescano a relazionarsi con touch-screen e computer.

Il presente documento descrive in dettaglio l'architettura logica, le scelte di design e l'implementazione software di **UNO FOCA**, un'applicazione desktop nata come riadattamento digitale del celebre gioco di carte collezionabili "UNO". L'obiettivo del progetto è coniugare le meccaniche di gioco tradizionali con un'identità visiva personalizzata in stile retro pixel art, strutturando al contempo un sistema di gestione dei dati leggero ed efficiente.

L'applicazione non si limita alla mera simulazione della partita, ma integra un modulo autonomo per la persistenza dei dati volto a valorizzare l'esperienza dell'utente attraverso:

- **Autenticazione Locale:** Un sistema di registrazione e accesso che identifica il giocatore e protegge il suo profilo tramite il file dedicato *player_info.bin*.
- **Tracciamento delle Performance:** un archivio locale (*statistiche.bin*) che monitora costantemente il bilancio di vittorie e sconfitte.
- **Storico dei Match:** una sezione di riepilogo in grado di mostrare i dati delle partite passate, garantendo un senso di progressione nel gioco.

Lo sviluppo del software è stato guidato da principi di modularità, ottimizzazione delle risorse (con una gestione rigorosa del caricamento e scaricamento dei dati in memoria) e totale indipendenza da server esterni, rendendo l'esperienza completamente fruibile in modalità offline. In questa documentazione si potranno trovare gli aspetti tecnici e grafici quanto anche schemi e tabelle che racchiuderanno il funzionamento complessivo del nostro lavoro accademico.

Complessivamente, per i vincoli che abbiamo avuto, la scrittura del programma è stata fatta in linguaggio C .

Le piattaforme usate e che consigliamo:

- GitHub per trovare strumenti utili, nel nostro caso la libreria Raylib ;
- Raylib , Canva , Gemini e Gimp per la grafica;
- Doxygen per parti della documentazione e formare una bozza iniziale ma particolarmente utile per la costruzione di diagrammi .

Gli sviluppatori

Abbamonte Giordano Alessandro

Messena Samuele

Sbano Sara Maria



1) Analisi

1.0 Richiesta del committente

Ci è stato richiesto di sviluppare un' applicazione che simuli una partita del gioco di carte UNO tra più giocatori in particolare tra bot e almeno uno che sia un utente reale. Il sistema deve gestire l'intero flusso della partita, dal mescolamento del mazzo fino alla determinazione del vincitore.

All'inizio il programma mescola il mazzo di carte e le distribuisce ai giocatori, scoprendo una prima carta. La partita per noi procede secondo le regole ufficiali delle carte UNO.

Durante il proprio turno, un giocatore può selezionare una carta valida da giocare rispetto a quella sul tavolo; in caso contrario, deve pescare dal mazzo. Il sistema deve verificare la correttezza delle mosse e aggiornare lo stato della partita costantemente.

Inoltre, il sistema dovrebbe fornire una rappresentazione chiara delle informazioni necessarie al giocatore per effettuare le proprie scelte.

I giocatori automatici possono adottare strategie diverse.

L'app inoltre includerà la possibilità di registrare un giocatore e associare a esso uno storico come il numero di partite giocate, le vittorie e altre statistiche utili a descriverne le prestazioni nel tempo.

Sarà possibile stoppare lo stato corrente della partita, in modo da poter interrompere il gioco e riprenderlo successivamente senza perdere informazioni. La partita termina quando un giocatore esaurisce le carte.

Il programma infine dichiara il vincitore, aggiorna le statistiche del giocatore registrato e poi consente l'avvio di una nuova partita o il caricamento di una partita salvata con tendina dedicata.

La conclusione del gioco non segue il classico andamento basato sul raggiungimento massimo di 500 punti ma bensì , come da richiesta, una volta finite tutte le carte del proprio mazzo.

1.1 Analisi delle specifiche

Il gioco UNO sviluppato è una nuova versione originale a tema foche con ambientazione ed elementi che rimandano al medesimo tema.

Il programma creato è sostanzialmente di *tipo Base*, quindi, presenta gli aspetti fondamentali del gioco ufficiale eccetto per la gestione della fine della partita, la variazione della difficoltà di gioco e infine il fatto che sia offline.

1.1.0 Carte

Nel contesto del programma e secondo le regole di gioco, abbiamo carte che si dividono per colori e funzioni speciali e con personaggi dedicati:

- Carte Gialle;
- Carte Rosse;
- Carte Blu;
- Carte Verdi;

numerate da 0-9, contribuiscono al botta e risposta fra utenti.

- Carte Speciali:
 - Carta +4 cambio colore
 - Carta cambio colore
 - Inverti giro
 - Blocco
 - Carte +2

1.1.1 Utente

L'utente avrà la possibilità di fare un login con nickname e password per poter salvare anche le proprie partite o avere un proprio storico.

1.1.2 Computer

L'utente affronterà giocatori bot e di bravura medio-alta per rendere la partita avvincente e incitare il giocatore a fare più partite.

1.2 Requisiti funzionali

Sono presenti tutte le funzionalità per raggiungere i requisiti previsti del programma con un codice identificativo e una descrizione per ogni funzione.

Tabella 1: MAIN.C - Sistema

Codice	Nome Funzione	Descrizione
F001	ScriviLogAccessoBinario	Scrive in modalità append ("ab") i log di tracciamento di login o registrazione dell'utente nel file binario registro_accessi.bin, filtrando l'account "admin".
F002	main	Punto di ingresso del gioco: inizializza la finestra, l'audio, alloca i moduli (Partita, Grafica, Menu), orchestra la macchina a stati ed esegue la pulizia finale della memoria.
F003	InitWindow (Raylib)	Crea e configura geometricamente la finestra principale dell'applicazione con una risoluzione di 1200x800 pixel e titolo personalizzato.
F004	SetTargetFPS (Raylib)	Sincronizza la frequenza di aggiornamento e la velocità del loop dell'applicazione impostandola stabilmente a 60 frame per secondo.

Codice	Nome Funzione	Descrizione
F005	InitAudio (<i>Sottosistema</i>)	Inizializza l'hardware del dispositivo audio di sistema e alloca i flussi della musica e degli effetti.
F006	LoadSound (<i>Raylib</i>)	Alloca in memoria audio la traccia sonora .mp3 utilizzata come feedback acustico per la pressione dei pulsanti.
F007	InizializzaGrafica	Carica in modo sicuro dal disco alla memoria video (VRAM) tutti gli asset grafici e le texture delle carte del tavolo da gioco.
F008	InizializzaMenu	Pulisce i buffer delle credenziali ed alloca le immagini di sfondo e i campi form legati ai moduli di login e registrazione.
F009	InizializzaMenuScelta	Configura geometricamente i rettangoli di collisione e carica le texture dei tasti per la selezione delle modalità (1vs1, Multiplayer, Impostazioni, Regole).
F010	InizializzaRisorseArchivio	Alloca i componenti grafici del menu archivio per permettere la consultazione dello storico e lo sblocco dei match ripresi.
F011	WindowShouldClose (<i>Raylib</i>)	Monitora in tempo reale se è stata richiesta la chiusura della finestra dall'utente tramite la barra di Windows.
F012	UpdateAudio (<i>Sottosistema</i>)	Esegue l'aggiornamento costante e periodico dei buffer di streaming della musica di sottofondo per evitarne l'interruzione.
F013	GetMousePosition (<i>Raylib</i>)	Campiona frame per frame le coordinate bidimensionali del cursore del mouse all'interno dello schermo per il calcolo delle collisioni.
F014	GestisceInput	Gestisce l'intercettazione dei clic e delle mosse regolamentari del giocatore umano sul tavolo da gioco.
F015	AggiornaPartita	Aggiorna i turni di gioco, applica gli effetti delle carte calate, processa l'IA dei bot e controlla lo sblocco dei deltapiani di notifica.

Codice	Nome Funzione	Descrizione
F016	SalvaPartitaSospesa	Esporta in modalità binaria lo stato completo della sessione di UNO corrente sul file di archivio in caso di uscita volontaria.
F017	StopBackgroundMusic	Interrompe bruscamente la riproduzione in streaming del file musicale di sottofondo durante i cambi di schermata.
F018	CheckCollisionPointRec (Raylib)	Funzione di intersezione geometrica utilizzata per valutare se il mouse si trova sopra le aree hitbox dei pulsanti.
F019	IsMouseButtonPressed (Raylib)	Rileva l'evento di clic del tasto sinistro del mouse per innescare le azioni e i cambi di stato associati ai pulsanti.
F020	PlaySound (Raylib)	Riproduce immediatamente l'effetto sonoro memorizzato del clic del mouse se l'audio è abilitato dal giocatore.
F021	OttieniStatisticheUtente	Interroga il database locale per estrarre e popolare il conteggio storico delle vittorie e delle sconfitte dell'utente registrato.
F022	ImpostaStatoMusica	Aggiorna il flag globale di sistema per abilitare o disabilitare l'esecuzione della musica d'atmosfera.
F023	ImpostaStatoEffetti	Aggiorna il flag globale per silenziare o attivare la riproduzione dei suoni delle carte e dei tasti di gioco.
F024	BeginDrawing (Raylib)	Configura il sistema grafico hardware per l'inizio del rendering del frame corrente a schermo.
F025	ClearBackground (Raylib)	Ripulisce l'intero schermo a ogni frame cancellando i residui del disegno precedente e impostando un colore nero di fondo.
F026	AggiornaDisegnaMenu	Gestisce la visualizzazione, il movimento del cursore e i clic sui tre bottoni del menu iniziale dell'applicazione.
F027	GestisciSchermataAuth	Gestisce i form di input per login e registrazione e, se rileva l'Admin con l'invio, apre il file registro_accessi.bin caricandone lo storico.

Codice	Nome Funzione	Descrizione
F028	AggiornaDisegnaMenuScelta	Gestisce la navigazione e restituisce l'azione di instradamento selezionata, nascondendo l'archivio se si accede come ospite.
F029	PlayBackgroundMusic	Avvia la traccia sonora musicale principale del tavolo da gioco all'inizio della partita.
F030	DisegnaPartita	Esegue il disegno del tavolo, delle carte disposte ad arco del giocatore, dei dorsi dei bot e delle animazioni attive.
F031	DisegnaArchivioStorico	Reinderizza l'interfaccia a griglia con lo storico delle partite sospese pre-caricate in memoria per consentirne il ripristino.
F032	CaricaPartiteSospeseUtente	Estrattore binario che recupera l'elenco strutturato dei vecchi salvataggi associati al profilo dell'utente.
F033	EliminaPartitaDaArchivio	Rimuove un salvataggio dal file locale una volta che il match è stato ripreso o sovrascritto con successo.
F034	DrawTexturePro (Raylib)	Disegna a schermo pulsanti o porzioni di immagini applicando rotazioni, scalature e filtri di colore di hover.
F035	MeasureText (Raylib)	Calcola in pixel la larghezza occupata da una stringa di testo per permetterne il perfetto centramento orizzontale sullo schermo.
F036	DrawText (Raylib)	Stampa stringhe di testo standard a schermo (es. il contatore numerico di vittorie in verde e sconfitte in rosso).
F037	DrawRectangle (Raylib)	Renderizza forme geometriche piene a schermo, utilizzate come sfondo scuro per il terminale amministrativo di debug.
F038	DrawLine (Raylib)	Disegna segmenti di linea retta colorata usati per separare graficamente le intestazioni dei record della tabella admin.
F039	EndDrawing (Raylib)	Conclude le operazioni di rendering e visualizza effettivamente a video sulla finestra il frame appena calcolato.

Codice	Nome Funzione	Descrizione
F040	UnloadSound (<i>Raylib</i>)	Disalloca in sicurezza dalla memoria RAM le risorse del file audio del clic prima dello spegnimento dell'app.
F041	ScaricaGrafica	Libera la memoria video (VRAM) rimuovendo tutte le texture delle carte e degli sfondi di gioco.
F042	ScaricaMenu	Rilascia le risorse delle immagini utilizzate all'interno delle schermate di autenticazione iniziale.
F043	DeinizializzaMenuScelta	Rilascia la memoria video associata alle immagini e agli sfondi del pannello di navigazione secondario.
F044	ScaricaRisorseArchivio	Libera la memoria VRAM dalle texture dello sfondo e delle frecce di navigazione del menu archivio.
F045	CloseAudio / CloseWindow	Disattiva definitivamente l'apparato hardware audio e chiude la finestra di gioco di Raylib terminando il programma.

Tabella 2: MENU.C – Accessi e Registrazione

Codice	Nome Funzione	Descrizione
F046	EsisteNickname	Verifica se un nickname è già presente nel file binario utenti.dat tramite lettura sequenziale ("rb").
F047	VerificaCredenzialiDettagliata	Convalida l'accesso confrontando nickname e password nel database, gestendo il bypass prioritario per l'account admin.
F048	RegistraNuovoUtenteInTabella	Scrive un nuovo record UtenteAccesso in modalità append ("ab") su utenti.dat, forzando il terminatore \0.
F049	AggiornaTimestampAccesso	Modifica in place ("rb+") il record dell'utente nel file binario, aggiornando la data e l'ora dell'ultimo accesso tramite fseek.
F050	SalvaLogStoricoAzione	Salva una traccia storica aggiuntiva delle operazioni (registrazione o login) nel file binario di log accessi_log.dat.
F051	GestisceInputTesto	Cattura i caratteri inseriti da tastiera, impedisce gli spazi nei campi password e gestisce la cancellazione con Backspace.
F052	GestisciSchermataAuth	Gestisce i form grafici di autenticazione e intercetta il tasto Invio per validare i dati o reindirizzare la macchina a stati.
F053	InizializzaMenu	Configura lo stato iniziale azzerando i buffer, disattiva il tasto di uscita di default (KEY_NULL) e carica le texture dei form.
F054	AggiornaDisegnaMenu	Calcola le posizioni dei pulsanti del menu principale e ne rileva le collisioni con il mouse per gestire l'hover e i clic.
F055	ScaricaMenu	Esegue la deallocazione completa dalla memoria video (VRAM) di tutte le texture e dei suoni associati al modulo.
F056	DisegnaTabellaDebug	Sottofunzione interna che renderizza graficamente la tabella dei log di accesso leggendo i record registrati in utenti.dat.

Tabella 3: MENU_SCELTA.C - Opzioni

Codice	Nome Funzione	Descrizione
F057	InizializzaMenuScelta	Alloca e azzerla la struttura di navigazione secondaria, carica dal disco tutte le texture di sfondo, le immagini dei pulsanti e i campioni sonori, impostando infine le dimensioni e le coordinate geometriche di ciascun elemento visivo.
F058	AggiornaDisegnaMenuScelta	Coordina a ogni frame il ciclo logico e di rendering del pannello. Genera le hitbox ridotte per massimizzare la precisione dei clic, verifica le intersezioni del mouse per gli effetti di hover e riproduce i suoni solo se abilitati; infine, occulta graficamente ed inibisce l'accesso all'archivio per gli utenti ospiti.
F059	DeinizializzaMenuScelta	Rilascia in modo sicuro le risorse allocate scaricando dalla memoria video (VRAM) tutte le texture degli sfondi e dei bottoni e disallocando il file audio del clic per evitare memory leak alla chiusura o al cambio di contesto.

Tabella 4: ARCHIVIO.C - Gestione storico partite

Codice	Nome Funzione	Descrizione
F060	PreparaArchivioStorico	Esegue il pre-caricamento (<i>caching</i>) dei salvataggi dal file binario alla RAM chiamando CaricaPartiteSospeseUtente per ottimizzare le prestazioni video.
F061	InizializzaRisorseArchivio	Carica dal disco le texture degli sfondi delle statistiche e dello storico, allocando geometricamente i rettangoli di collisione delle frecce di navigazione.
F062	ScaricaRisorseArchivio	Disalloca in sicurezza dalla memoria video (VRAM) tutte le texture caricate per la schermata dello storico e del pannello delle statistiche.
F063	SalvaPartitaSospesa	Registra su file binario lo stato del match inserendo un timestamp. Applica la logica FIFO per limitare a un massimo di 5 i salvataggi simultanei per ciascun utente.
F064	CaricaPartiteSospeseUtente	Svolge una scansione binaria inversa su salvataggi.dat per estrarre e ordinare, dal record più recente a quello più vecchio, i match salvati dall'utente.
F065	EliminaPartitaDaArchivio	Rimuove selettivamente dal database locale un record di salvataggio specifico a seguito del ripristino o della sovrascrittura della sessione.
F066	DisegnaArchivioStorico	Renderizza la griglia visiva dei match sospesi attingendo alla cache in RAM, gestisce l'effetto di hover e restituisce l'indice del record selezionato dal mouse.
F067	SalvaRisultatoPartita	Accede al file statistiche_utenti.dat per incrementare e sovrascrivere in modo permanente il contatore delle vittorie o delle sconfitte a fine match.
F068	OttieniStatisticheUtente	Svolge una ricerca sequenziale nel database binario delle statistiche per prelevare il numero totale di vittorie e sconfitte abbinate al giocatore.

Tabella 5: GAME.C

Codice	Nome Funzione	Descrizione
F069	OttieniPosizioneMazzoGiocatore	Restituisce le coordinate vettoriali bidimensionali sullo schermo adibite al posizionamento grafico del mazzo delle carte dei giocatori umani o dei bot, differenziandole in base alla modalità (1vs1 o 1vs3).
F070	MescolaArray	Routine interna statica che randomizza e scambia in modo casuale la posizione degli elementi all'interno di un array di strutture carte mediante l'uso di indici casuali.
F071	CreaMazzoCompleto	Genera l'intera struttura logica iniziale delle 108 carte regolamentari del mazzo di UNO, suddividendole per colori (da 0 a 9, Salta, Inverti, +2) e includendo le carte speciali nere (+4 e Cambio Colore).
F072	RigeneraMazzo	Sposta in blocco le carte accumulate nella pila degli scarti all'interno del mazzo di pesca principale e le rimescola nel caso in cui il mazzo di gioco si esaurisca completamente.
F073	Pesca	Preleva la carta successiva situata in cima alla pila del mazzo di pesca incrementando l'indice; invoca in via condizionale la rigenerazione automatica se le carte disponibili sono esaurite.
F074	ImpostaMessaggio	Copia in modo sicuro una stringa di testo all'interno dell'interfaccia utente di gioco e inizializza un timer float per visualizzarla a schermo stabilmente per 3.5 secondi.
F075	ApplicaPenalita	Costringe un determinato giocatore bersaglio o bot a prelevare sequenzialmente dal mazzo di pesca una quantità fissa di carte sanzionatorie dovute agli attacchi degli avversari.

Codice	Nome Funzione	Descrizione
F076	PuoGiocare	Verifica se una determinata carta selezionata dalla mano può essere calata sul tavolo rispettando i vincoli del regolamento (corrispondenza di colore, tipo, valore o se si tratta di carta nera).
F077	GiocatoreHaMosse	Esegue una scansione completa di tutte le carte presenti all'interno della mano di uno specifico giocatore per valutare se possiede almeno un'azione valida da calare sul tavolo.
F078	InizializzaPartita	Azzera e predispone lo stato logico della sessione: configura i profili e le strategie IA dei bot, mescola il mazzo principale, distribuisce 7 carte a ogni partecipante e gestisce gli effetti della prima carta scartata sul tavolo.
F079	SalvaPartita	Registra e scrive in append binario ("ab") lo stato logico completo del match all'interno del file locale salvataggi_partite.dat, includendo un timestamp temporale (se il profilo non è ospite).
F080	ProcessaCartaGiocata	Esegue l'applicazione matematica degli effetti delle carte speciali giocate sul tavolo, modificando la direzione di giro dei turni, applicando i malus (+2/+4) e abilitando i flag di selezione colore.
F081	GestisceInput	Intercetta e decodifica i comandi e i clic del mouse dell'utente umano per gestire la selezione delle carte, la pescata singola regolamentare, il passaggio condizionale del turno o la sospensione della partita.
F082	SceltaStrategica	Algoritmo di intelligenza artificiale per i bot a difficoltà difficile: assegna una priorità difensiva che induce a consumare prima le carte numeriche base, conservando le speciali per i turni avanzati.

Codice	Nome Funzione	Descrizione
F083	AggiornaPartita	Sottosistema centrale che coordina il ciclo del match: gestisce le tempistiche dei bot (0.8s di attesa), avvia i movimenti animati delle carte e del deltaplano, e valuta le condizioni di vittoria o sconfitta a fine match.

Tabella 6: GRAPHICS.C – grafica e stile

Codice	Nome Funzione	Descrizione
F084	CreaFallback	Genera programmaticamente in memoria un'immagine geometrica rettangolare di colore grigio scuro, convertendola in texture per prevenire crash in caso di asset mancanti.
F085	CaricaSicuro	Verifica l'esistenza fisica sul disco di un file d'immagine tramite Raylib prima di effettuarne il caricamento; in caso di assenza, restituisce la texture di fallback.
F086	DisegnaCarta	Renderizza a schermo una singola carta applicando le coordinate cartesiane di destinazione, le dimensioni regolamentari e l'angolo specifico di inclinazione.
F087	EvidenziaCarta	Disegna un rettangolo colorato di contorno leggermente più grande dietro la carta per evidenziare visivamente che è selezionata o giocabile dal utente.
F088	DisegnaTestoPixelGrassetto	Crea un effetto di testo retro pixel art in grassetto applicando un algoritmo ad alto contrasto che disegna la stringa in nero lungo più direzioni prima di sovrapporvi il colore base.
F089	OttieniCarta	Esamina le proprietà logiche di una carta (colore, tipo, valore) e restituisce il relativo puntatore alla texture corretta memorizzata all'interno della matrice grafica.
F090	InizializzaGrafica	Azzera la struttura del modulo e alloca in memoria VRAM tutti gli sfondi, i bottoni, le schermate di fine partita e cicla i file del disco per mappare l'intero set di carte.
F091	DisegnaPartita	Coordina l'intero rendering del match: dispone ad arco le carte del giocatore umano, gestisce la disposizione dei mazzi dei bot per modalità 1vs1 o 1vs3, e disegna i pop-up e le animazioni.

Codice	Nome Funzione	Descrizione
F092	ScaricaGrafica	Esegue la deallocazione sicura e sistematica dalla memoria video di tutti gli asset visivi del modulo e delle carte da gioco, escludendo i fallback già rimossi per evitare leak.

Tabella 7: AUDIO.C- suoni ed effetti

Codice	Nome Funzione	Descrizione
F093	InitAudio	Attiva il dispositivo audio hardware e alloca in memoria RAM i file multimediali dedicati alla colonna sonora (.mp3) e agli effetti sonori del tavolo da gioco, inizializzando i flag di attivazione su true.
F094	PlayBackgroundMusic	Avvia la riproduzione in streaming ciclico del brano musicale di sottofondo principale, previo controllo condizionale sullo stato del flag locale di attivazione della musica.
F095	UpdateAudio	Aggiorna frame per frame i buffer della traccia musicale attiva per assicurarne uno streaming continuo e fluido durante la partita, a patto che la musica non sia disabilitata.
F096	StopBackgroundMusic	Comanda l'interruzione immediata e l'arresto del flusso di riproduzione del brano musicale di sottofondo attualmente in esecuzione tramite le librerie hardware.
F097	PlayCardSound	Riproduce istantaneamente il campione sonoro associato al posizionamento o alla calata di una carta sul tavolo da gioco, solo se gli effetti sonori risultano abilitati dal giocatore.
F098	PlayWinSound	Riproduce immediatamente l'effetto sonoro celebrativo della vittoria a fine partita, subordinatamente alla verifica dello stato di attivazione globale degli effetti sonori.
F099	PlayDefeatSound	Innesca la riproduzione immediata del file audio correlato alla sconfitta del giocatore contro l'intelligenza artificiale, se il sottosistema degli effetti non è silenziato.
F100	ImpostaStatoMusica	Configura e aggiorna il flag di stato booleano della traccia d'ambiente; se impostato su false, provvede a bloccare all'istante l'esecuzione di qualsiasi brano musicale attivo.

Codice	Nome Funzione	Descrizione
F101	ImpostaStatoEffetti	Aggiorna dinamicamente il flag booleano globale di controllo degli effetti sonori per abilitare o silenziare i suoni delle carte, dei pulsanti e dei pop-up.
F102	CloseAudio	Esegue le operazioni terminali di pulizia e deallocazione, scaricando definitivamente i flussi multimediali e spegnendo in sicurezza la scheda audio per prevenire leak di memoria.

1.3 Casi d'uso

I casi d'uso sono in riferimento alle modalità con cui si possono presentare le funzioni previste dal programma già sopra riportate. Un requisito può essere associato a più casi d'uso quindi un insieme di funzioni distinte portano a specifici utilizzi .

La tabella prevede :

- PRE-CONDIZIONI , eventi che scatenano l'uso di una funzione;
- POST-CONDIZIONI , con relative condizioni nel caso SUCCESSI e FALLIMENTI;
- SCENARIO DI BASE , quello generato in caso di successo;
- SCENARIO ALTERNATIVO, quello generato in caso di fallimento

Caso d'uso 1: Inizializzazione del Gioco e Setup Finestra

Elemento	Specifica
Codice Funzioni	F002 - F003 - F004
Nome Caso d'uso	Inizializzazione del Gioco e Setup Finestra
Descrizione	Il programma deve configurare l'ambiente grafico all'avvio dell'applicazione, impostando la risoluzione e il target del framerate.
Pre-condizioni	1. L'eseguibile viene lanciato dall'utente. 2. Le librerie grafiche (Raylib) sono collegate correttamente.
Post-condizioni (Successo)	La finestra di gioco è aperta stabilmente a 1200x800 pixel, con titolo personalizzato e loop sincronizzato a 60 FPS.
Post-condizioni (Fallimento)	L'applicazione crasha all'avvio o la finestra si chiude immediatamente per problemi di contesto grafico.

Elemento	Specifica
Scenario di Base	L'utente avvia l'applicazione. Il main (F002) invoca InitWindow (F003) impostando le dimensioni geometriche e il titolo. Successivamente invoca SetTargetFPS (F004) per bloccare la frequenza di aggiornamento a 60 frame al secondo, istanziando la macchina a stati sul menu principale.
Scenario Alternativo	<i>Inizializzazione fallita:</i> Le risorse video del sistema non supportano la modalità. Raylib lancia un errore su terminale e il main termina forzatamente restituendo un codice d'errore.

Caso d'uso 2: Inizializzazione Sottosistema Audio e Caricamento Asset

Elemento	Specifica
Codice Funzioni	F002 - F005 - F006 - F012
Nome Caso d'uso	Inizializzazione Sottosistema Audio e Caricamento Asset
Descrizione	Il programma deve inizializzare l'hardware audio e allocare in memoria i flussi sonori e gli effetti d'interazione.
Pre-condizioni	1. Il gioco è in fase di avvio (F002). 2. I file audio (es. .mp3 per i pulsanti) sono presenti nella cartella delle risorse.
Post-condizioni (Successo)	L'hardware audio è allocato e i buffer per la musica e gli effetti del clic sono pronti in memoria.
Post-condizioni (Fallimento)	Il gioco si avvia in modalità muta o notifica l'assenza della scheda audio.
Scenario di Base	Durante l'esecuzione del main, viene chiamato il sottosistema audio (F005) che configura il dispositivo hardware. Subito dopo, viene invocata LoadSound (F006) per caricare in memoria l'effetto acustico del clic del mouse. Durante tutto il ciclo vitale, UpdateAudio (F012) aggiorna costantemente i buffer dello streaming musicale per evitarne l'interruzione.

Elemento	Specifica
Scenario Alternativo	<i>Dispositivo audio assente:</i> Il sistema non rileva hardware audio valido. Disabilita internamente i flag di riproduzione per evitare crash e prosegue l'avvio del gioco in modalità muta. <i>File audio mancante:</i> Il file .mp3 non viene trovato sul disco. Il sistema assegna un puntatore nullo alla risorsa senza bloccare l'esecuzione globale.

Caso d'uso 3: Caricamento Asset Grafici e Moduli Interfaccia

Elemento	Specifica
Codice Funzioni	F002 - F007 - F008 - F009 - F010
Nome Caso d'uso	Caricamento Asset Grafici e Moduli Interfaccia
Descrizione	Il programma deve allocare in memoria video (VRAM) e inizializzare i moduli legati alla grafica di gioco, del menu di login, della scelta modalità e dell'archivio storico.
Pre-condizioni	1. La finestra Raylib è aperta correttamente. 2. Le texture delle carte del tavolo da gioco e gli sfondi sono disponibili su disco.
Post-condizioni (Successo)	Tutti i moduli (Partita, Grafica, Menu) sono allocati in modo sicuro e le texture sono caricate in VRAM.
Post-condizioni (Fallimento)	Mancano texture critiche, il sistema mostra artefatti visivi o crasha per violazione di memoria.
Scenario di Base	Il main (F002) chiama in sequenza le funzioni di setup. InizializzaGrafica (F007) carica gli asset delle carte in VRAM. InizializzaMenu (F008) pulisce i campi credenziali e carica gli sfondi di login/registrazione. InizializzaMenuScelta (F009) configura i rettangoli di collisione (hitbox) delle modalità (1vs1, Multiplayer, ecc.) e InizializzaRisorseArchivio (F010) alloca la grafica per consultare lo storico dei match.

Elemento	Specifica
Scenario Alternativo	<i>Memoria video insufficiente / Asset corrotti</i> : Il sistema rileva l'impossibilità di caricare una texture. Viene generato un log di errore e l'applicazione carica un placeholder grafico d'emergenza o termina l'esecuzione pulendo la memoria già allocata.

Caso d'uso 4: Tracciamento Log Accessi Utente

Elemento	Specifica
Codice Funzioni	F001
Nome Caso d'uso	Tracciamento Log Accessi Utente
Descrizione	Il programma deve registrare in modo persistente in un file binario ogni operazione di login o registrazione, ignorando i tracciamenti legati all'account amministratore.
Pre-condizioni	1. Un utente sta effettuando un tentativo di login o una nuova registrazione nel modulo Menu (F008). 2. Il sistema ha i permessi di scrittura sul disco locale per il file registro_accessi.bin.
Post-condizioni (Successo)	L'evento di accesso viene appeso in modalità binaria nel file di registro se l'utente non è "admin".
Post-condizioni (Fallimento)	Il log non viene scritto, ma l'operazione di gioco prosegue senza bloccare l'utente.
Scenario di Base	Un utente comune inserisce le credenziali e preme il tasto di conferma. Il sistema valida l'operazione e invoca ScriviLogAccessoBinario (F001). La funzione verifica che la stringa dell'account non sia "admin", apre il file registro_accessi.bin in modalità append ("ab") e scrive la struttura dati del log prima di chiudere il descrittore del file.

Elemento	Specifica
Scenario Alternativo	<i>Accesso Account Admin:</i> Se l'utente che effettua l'accesso ha come username "admin", la funzione intercetta il filtro di sicurezza ed esce immediatamente senza effettuare alcuna scrittura sul file binario. <i>File bloccato o senza permessi:</i> Se il sistema operativo nega l'accesso in scrittura al file, la funzione fallisce silenziosamente per non interrompere l'esperienza dell'utente.

Caso d'uso 5: Gestione Input Mouse e Selezione Elementi

Elemento	Specifica
Codice Funzioni	F013 - F014 - F018 - F019 - F020
Nome Caso d'uso	Gestione Input Mouse e Selezione Elementi
Descrizione	Il programma deve intercettare la posizione del mouse e i clic del giocatore umano per calcolare le collisioni con i pulsanti e le carte del tavolo da gioco, restituendo un feedback sonoro.
Pre-condizioni	1. Il gioco si trova nello stato di Partita o Menu attivo. 2. I rettangoli di collisione (hitbox) degli elementi grafici sono stati configurati geometricamente (F009).
Post-condizioni (Successo)	Il clic viene registrato, l'azione corrispondente (es. calare una carta o premere un tasto) viene eseguita e viene riprodotto il suono del clic.
Post-condizioni (Fallimento)	L'input viene ignorato o le coordinate del mouse non corrispondono alla reale posizione sullo schermo.

Elemento	Specifica
Scenario di Base	Frame per frame, GetMousePosition (F013) campiona le coordinate bidimensionali del cursore. Quando l'utente preme il tasto sinistro, IsMouseButtonPressed (F019) intercetta l'evento. GestisceInput (F014) coordina il controllo e usa CheckCollisionPointRec (F018) per valutare se il mouse si trova sopra la hitbox di un pulsante o di una carta. Se c'è collisione, viene eseguito il comando e PlaySound (F020) riproduce il feedback acustico del clic.
Scenario Alternativo	<i>Clic fuori dalle Hitbox:</i> L'utente clicca in un'area vuota dello schermo. CheckCollisionPointRec restituisce <i>false</i> , l'input viene scartato, non viene riprodotto alcun effetto sonoro e lo stato del gioco non subisce variazioni.

Caso d'uso 6: Aggiornamento Stato Partita e Salvataggio Sessione

Elemento	Specifica
Codice Funzioni	F015 - F016 - F021
Nome Caso d'uso	Aggiornamento Stato Partita e Salvataggio Sessione
Descrizione	Il sistema deve orchestrare i turni di gioco, applicare gli effetti delle carte, processare l'Intelligenza Artificiale dei bot, gestire le notifiche e permettere il salvataggio binario della partita in caso di uscita volontaria.
Pre-condizioni	1. Una sessione di gioco è avviata e allocata in memoria. 2. L'utente ha un profilo registrato per l'estrazione delle statistiche storiche.
Post-condizioni (Successo)	Lo stato della partita avanza coerentemente; in caso di interruzione, la sessione viene esportata su file binario senza perdita di dati.
Post-condizioni (Fallimento)	Mancata sincronizzazione del turno o corruzione del file di salvataggio.

Elemento	Specifica
Scenario di Base	Il loop di gioco invoca costantemente AggiornaPartita (F015), che incrementa i turni, elabora le mosse dei bot della CPU e controlla lo sblocco dei deltapiani di notifica su schermo. All'avvio del match, il sistema usa OttieniStatisticheUtente (F021) per popolare lo storico di vittorie/sconfitte. Se il giocatore sceglie di uscire anzitempo, viene chiamata SalvaPartitaSospesa (F016) per esportare lo stato completo del tavolo in formato binario nell'archivio.
Scenario Alternativo	<i>Uscita senza salvataggio (Abbandono forzato):</i> Se il gioco viene chiuso bruscamente tramite la barra del sistema operativo senza passare dal menu di pausa, SalvaPartitaSospesa non viene invocata. Al riavvio successivo, il match non potrà essere ripreso e verrà considerato come una sconfitta nelle statistiche estratte da OttieniStatisticheUtente.

Caso d'uso 7: Terminazione dell'Applicazione e Pulizia Memoria

Elemento	Specifica
Codice Funzioni	F002 - F011 - F017
Nome Caso d'uso	Terminazione dell'Applicazione e Pulizia Memoria
Descrizione	Il programma deve intercettare la richiesta di chiusura della finestra, interrompere i flussi audio e liberare tutta la memoria precedentemente allocata nel modulo di sistema.
Pre-condizioni	L'applicazione è in esecuzione all'interno del loop principale nel main.
Post-condizioni (Successo)	La musica viene interrotta, la memoria heap (moduli partita, grafica, menu) viene liberata correttamente, la finestra viene chiusa e il programma termina con codice 0 (nessun memory leak).
Post-condizioni (Fallimento)	Il programma si blocca in chiusura (zombie process) o lascia porzioni di memoria RAM/VRAM occupate (memory leak).

Elemento	Specifica
Scenario di Base	Durante l'esecuzione del loop, WindowShouldClose (F011) monitora se l'utente ha cliccato la "X" della finestra di sistema. Quando l'evento restituisce vero, il ciclo principale si interrompe. Il programma invoca StopBackgroundMusic (F017) per stoppare lo streaming audio e il main (F002) esegue la pulizia finale esplicita della memoria disallocando i puntatori dei moduli (Partita, Grafica, Menu) prima di restituire il controllo al sistema operativo.
Scenario Alternativo	<i>Chiusura da Task Manager / Segnale di Terminate:</i> Se il processo viene interrotto bruscamente dall'esterno tramite segnali del sistema operativo (es. SIGKILL), l'esecuzione ordinata del main viene saltata. La memoria heap viene comunque rilasciata dal sistema operativo moderno, ma i file di log o i salvataggi in corso potrebbero rimanere incompleti o inconsistenti.

1. Utente

- **Salvataggio Locale:** Il nome viene memorizzato nelle preferenze dell'app per essere mostrato durante le partite e nelle statistiche.
- **Registrarsi o Creare un profilo:** L'utente crea il proprio profilo con nome e password associata per un max di 10 caratteri ciascuno e non possono essere modificati successivamente; se errati il sistema non permette l'accesso e avvisa l'utente con una scritta.
- **Accesso:** l'utente se già registratosi può inserire le credenziali e il sistema rileverà la corrispondenza nel database.
- **Accedere come Ospite:** l'utente può accedere senza autenticarsi.
- **Salvataggio partita** e caricamento in un momento secondario visualizzabile nell'Archivio;
- **Regolamento consultabile:** l'utente può cliccare sul bottone del regolamento ed essere indirizzato con il link del regolamento ufficiale del gioco UNO.

2. Gestione della partita

- **Single o multiplayer:** L'utente sceglie come desidera giocare tra la modalità 1v1 o 1v3 ; Il sistema poi lo abbina a bot di media difficoltà standardizzata dagli sviluppatori.
- **Giocare una Carta:** L'utente seleziona una carta dalla propria mano. Il sistema convalida se la carta è giocabile (stesso colore, numero o simbolo) rispetto a quella sul mazzo degli scarti.
- **Carta consigliata:** il sistema rende possibile all'utente di visualizzare le carte da poter giocare in quel momento; tale suggerimento è evidenziato dalle carte poste leggermente più in alto rispetto alle altre del proprio mazzo.
- **Pescare una Carta:** il sistema permette di pescare in automatico e subito un'altra carta senza che lo faccia l'utente manualmente . Alternativamente, l'utente è libero di pescare anche manualmente le carte.
- **Utilizzo Carte Speciali:** L'utente gioca un "+2", un "Inverti" o un "Salto". Il sistema applica l'effetto al giocatore successivo o cambia il senso di rotazione e il sistema avvisa il giocatore.
- **Dichiarare "UNO!":** Quando l'utente rimane con una sola carta, viene dichiarato "UNO".

3. Dinamiche di Gioco contro l'IA

- **Gestione Turni Bot:** Il sistema calcola la mossa del bot. Viene inserito un **ritardo artificiale** per permettere all'utente di capire cosa sta succedendo.
- **Dichiarazione "UNO" (User):** L'utente dichiara "UNO" tramite un fumetto.
- **Dichiarazione "UNO" (Bot):** Il bot dichiara "UNO" tramite un fumetto.

4. Funzioni dedicate agli sviluppatori

- **Monitoraggio utenti:** gli sviluppatori , con nickname "admin" e password "1234", possono accedere ad una tabella sommatoria degli accessi e dei login fatti dai giocatori con rispettivi orari e date.

Le registrazioni e i login saranno riconoscibili nella tabella grazie alle iniziali "R" o "L".

2) Progettazione

Questo progetto implementa un gioco UNO in C usando la libreria Raylib per grafica, input e audio. Il programma offre:

- menu iniziale con accesso, registrazione e modalità ospite;
- gioco UNO contro bot in modalità 1vs1 oppure 1vs3;
- schermata regole, impostazioni audio/musica e archivio salvataggi;
- salvataggio e ripresa delle partite per utenti registrati;
- statistiche di vittorie e sconfitte;
- pannello amministratore per visualizzare i log binari di accesso.

La finestra di gioco ha dimensione fissa 1200x800 e gli asset sono caricati dalle cartelle Sfondo, Pulsanti, Carte, Animazioni e Musiche.

Tecnologie e dipendenze:

Il progetto è scritto in C e viene compilato con CMake.

Dipendenza principale:

- Raylib

Come indicato in CMakeLists.txt, la libreria può essere installata con:

sudo dnf install raylib-devel

Il target viene linkato anche con librerie di sistema usate da Raylib su Linux:

- GL
- m
- pthread
- dl
- rt
- X11

Compilazione ed esecuzione:

Dalla directory principale del progetto:

```
cmake -S . -B cmake-build-debug
cmake --build cmake-build-debug
./cmake-build-debug/UNO
```

Il file CMakeLists.txt copia automaticamente nella directory di build le cartelle degli asset:

- Musiche
- Sfondo
- Animazioni
- Pulsanti
- Carte

Questo passaggio è necessario perché il gioco carica immagini e suoni usando percorsi relativi.

2.0.1 Librerie e funzioni

Struttura del nostro progetto

```
.
├── main.c          # Ciclo principale, macchina degli stati e coordinamento moduli
├── game.c/game.h  # Regole UNO, stato partita, bot, turni e animazioni logiche
├── graphics.c/.h  # Caricamento texture e rendering della partita
├── audio.c/.h     # Musica, effetti sonori e mute/unmute
├── menu.c/.h      # Menu iniziale, login, registrazione e gestione utenti
├── menu_scelta.c/.h # Menu dopo login/ospite: modalita, regole,archivio,impostazioni
├── archivio.c/.h  # Salvataggi, ripresa partite e statistiche utente
├── CMakeLists.txt  # Configurazione build
├── Carte/         # Texture delle carte UNO
├── Sfondo/        # Sfondi delle schermate
├── Pulsanti/      # Immagini dei pulsanti
├── Animazioni/    # Immagini finali e popup
└── Musiche/       # Musica ed effetti audio
```

Il programma contiene le seguenti librerie di C:

Libreria .h	Tipo di Libreria	Scopo e utilizzo nel Progetto
<stdio.h>	Standard C	Gestione delle operazioni di Input/Output. Viene utilizzata per la formattazione avanzata delle stringhe (tramite sprintf) e per la stampa a schermo di messaggi di errore ed avvisi nel terminale.
<stdlib.h>	Standard C	Gestione delle funzioni di utilità generale, allocazione dinamica e conversioni.
<string.h>	Standard C	Manipolazione e gestione delle stringhe di testo. È fondamentale nel modulo di autenticazione per copiare i dati in modo sicuro (tramite strncpy) e confrontare nickname e password (tramite strcmp).
<stdbool.h>	Standard C	Introduzione del tipo di dato nativo booleano (bool) e delle costanti logiche true e false, usate per gestire i flag di stato del gioco (es. se l'utente è un ospite o se una risorsa è nascosta).
<time.h>	Standard C	Gestione del tempo e delle date di sistema. Viene utilizzata per generare un timestamp preciso (data e ora locali tramite localtime e strftime) registrato nei file binari di log accessi e salvataggi.
<math.h>	Standard C	Funzioni matematiche comuni, inclusa per supportare i calcoli geometrici correlati al disegno, posizionamento e rotazione delle carte sullo schermo.
"raylib.h"	Esterna (Grafica, Input & Audio)	Gestisce interamente l'inizializzazione della finestra, il ciclo di rendering delle texture, il disegno del testo pixel art, la riproduzione di file audio (.mp3) e il rilevamento degli input di mouse e tastiera.

Oltre alle librerie di sistema sopra elencate, il codice si affida a File Header aggiuntivi come:

- **"game.h"**: contiene le strutture dati principali come Carta, Giocatore, Partita e le funzioni che regolano il flusso di gioco e la persistenza dei salvataggi binari.
- **"graphics.h"**: mappa la struttura Grafica adibita a caricare e deallocare in modo sicuro le texture dal disco alla memoria video.
- **"audio.h"**: dichiara le routine di attivazione, aggiornamento e rilascio dello streaming musicale e degli effetti sonori del gioco.
- **"menu.h"** e **"menu_scelta.h"**: regolano i menu di navigazione, i form dei campi di input testo e le routine del pannello di debug per l'amministratore.
- **"archivio.h"**: Il suo scopo principale è permettere ai giocatori registrati di salvare lo stato delle partite, consultare lo storico dei match passati e monitorare il proprio bilancio di vittorie e sconfitte, ottimizzando al contempo l'accesso al disco e la memoria video. Il limite massimo di file di salvataggio (partite sospese) che ogni singolo utente può memorizzare contemporaneamente nel database locale è di 5.

Funzioni API della Libreria Raylib:

- **Gestione Finestra e Tempo:** `InitWindow` , `SetTargetFPS` , `WindowShouldClose` , `CloseWindow` , `GetFrameTime` .
- **Gestione Rendering e Figure:** `BeginDrawing` , `EndDrawing` , `ClearBackground` , `DrawTexturePro` , `DrawTextureEx` , `DrawRectangle` , `DrawRectanglePro` , `DrawRectangleRec` , `DrawRectangleLinesEx` , `DrawRectangleRounded` , `DrawRectangleRoundedLines` , `Fade` .
- **Caricamento e Manipolazione Asset Grafici:** `LoadTexture` , `LoadTextureFromImage` , `UnloadTexture` , `GenImageColor` , `UnloadImage` , `FileExists` .

- **Gestione Testo e Font:** `DrawText` , `MeasureText`.
- **Gestione Input Tastiera e Mouse:** `IsKeyPressed` , `GetCharPressed` , `GetMousePosition` , `CheckCollisionPointRec` , `IsMouseButtonPressed` , `SetExitKey`.
- **Gestione Audio e Flussi Sonori:** `InitAudioDevice` , `CloseAudioDevice` , `LoadMusicStream` , `UnloadMusicStream` , `PlayMusicStream` , `UpdateMusicStream` , `StopMusicStream` , `IsMusicStreamPlaying` , `LoadSound` , `UnloadSound` , `PlaySound`.
- **Generazione Numeri Casuali:** `GetRandomValue`.

2.0.2 Gestione e registrazione utente

Autenticazione

Funzioni interne principali:

- **EsisteNickname(...):** controlla se un nickname è già presente;
- **VerificaCredenzialiDettagliata(...):** verifica login e riconosce l'admin;
- **RegistraNuovoUtenteInTabella(...):** aggiunge un nuovo utente;
- **AggiornaTimestampAccesso(...):** aggiorna ultimo accesso dell'utente;
- **SalvaLogStoricoAzione(...):** aggiunge un record allo storico.

Credenziali amministratore:

- **username:** admin
- **password:** 1234

Con queste credenziali il programma entra in `STATO_DEBUG`, dove `MAIN.C` legge **registro_accessi.bin** e mostra una tabella di registrazioni e login.

2.0.3 Gestione partita e storico partita

- In GAME.H

```
91 // Funzioni di ciclo e logica di gioco
92 void InizializzaPartita(Partita *p, int num_giocatori);
93 void AggiornaPartita(Partita *p);
94 void GestisceInput(Partita *p);
95 bool PuoGiocare(Carta c, Partita *p);
96 bool GiocatoreHaMosse(Partita *p, int id_giocatore);
97 Carta Pesca(Partita *p);
```

- **InizializzaPartita(Partita *p):** Resetta lo stato del match, pulisce la struttura dati, genera il mazzo da 108 carte, lo mescola e distribuisce le mani iniziali a giocatore e bot.
- **AggiornaPartita(Partita *p):** Gestisce lo scorrimento dei turni, l'intelligenza artificiale del bot, i timer dei messaggi, le animazioni e controlla se si sono verificate le condizioni di vittoria o sconfitta.
- **GestisceInput(Partita *p):** Cattura i clic del mouse del giocatore per permettergli di selezionare una carta dalla mano, pescare dal mazzo o premere il pulsante per sospendere la partita.
- **PuoGiocare(Carta c, Partita *p):** Controlla i regolamenti di UNO per stabilire se una carta in mano al giocatore può essere scartata sopra quella attualmente in cima.
- **GiocatoreHaMosse(Partita *p):** Controlla il giocatore per capire se ha almeno una mossa valida a disposizione o se è costretto a pescare.
- **Pesca(Partita *p):** Estrae la carta successiva dal mazzo di pesca e sposta in avanti l'indice di lettura.

2.0.4 Mappatura globale delle strutture dati, costanti e file di persistenza del progetto

Nome	Tipologia	Descrizione	Tipi / Campi / Valore
utenti.dat	File Binario	Database locale contenente l'anagrafica e le credenziali cifrate/standardizzate degli utenti registrati.	Struttura dei record: UtenteRegistrato
accessi_log.dat	File Binario	Registro cronologico in modalità <i>append</i> degli eventi di login e registrazione del sistema.	Struttura dei record: LogAccesso
statistiche.dat	File Binario	Database in cui vengono salvati i dati storici delle partite di ogni utente (vittorie, sconfitte, match giocati).	Struttura dei record: StatisticheUtente
UtenteRegistrato	Struttura Dati (struct)	Record dell'utente memorizzato nel file binario del database (menu.h).	nickname (char[11])password(char[11])ultimo_accesso(char[25])ultima_azione (int)
LogAccesso	Struttura Dati (struct)	Record di tracciamento per lo storico delle azioni di sicurezza (menu.c).	nickname (char[11])password (char[11])data_oro(char[25])tipo_azione (int)
StatisticheUtente	Struttura Dati (struct)	Record contenente lo storico dei match e i punteggi per il modulo dell'archivio (archivio.h).	nickname (char[11])partite_giocate (int)vittorie(int)sconfitte (int)punti_totali (int)

Carta	Struttura Dati (struct)	Modello atomico di una singola carta da gioco del mazzo (game.h).	valore (int / Enum: 0-9, +2, Cambio Giro, Salto Turno, Jolly, +4) colore (int / Enum: ROSSO, GIALLO, VERDE, BLU, NERO) id (int, chiave univoca dello sprite)
Giocatore	Struttura Dati (struct)	Rappresentazione logica dell'entità partecipante (Umano o Bot AI) (game.h).	nickname (char[11]) mano (Carta[MAX_CARTE_MANO]) num_carte (int) is_bot (bool)
StatoPartita	Struttura Dati (struct)	Contesto globale del motore logico della partita attiva (game.h).	giocatori (Giocatore[MAX_GIOCATORI]) mazzo_pesca (Carta[108]) top_mazzo_pesca (int) mazzo_scarto (Carta[108]) top_mazzo_scarto (int) turno_corrente (int) direzione (int: 1 o -1) colore_corrente (int) fase_gioco (int)
MenuGrafica	Struttura Dati (struct)	Contesto dati, buffer di input e texture per le schermate di login e registrazione (menu.h).	sfondo, sfondo_accedi, sfondo_registrati (Texture2D) btn_accedi, btn_registrati, btn_ospite (Texture2D) tex_utente, tex_password (Texture2D) nickname (char[12]) password (char[12]) campo_attivo (int) frame_counter (int)
MenuScelta	Struttura Dati (struct)	Gestore geometrico e delle risorse grafiche per l'hub di selezione delle modalità (menu_scelta.h).	sfondo, sfondoImpostazioni, sfondoArchivio, sfondoRegole (Texture2D) btn1vs1, btnMulti, btnRegole, btnImpostazioni, btnArchivio, btnIndietro (Texture2D) btnOnMusica, btnOnAudio, btnOff (Texture2D) rect1vs1, rectMulti, rectRegole, rectArchivio, rectImpostazioni, rectIndietro (Rectangle) musicaAttiva, audioAttivo, nascondi_archivio (bool)

ArchivioGrafica	Struttura Dati (struct)	Gestore delle risorse grafiche e logiche per la navigazione impaginata della cronologia (archivio.h).	sfondo (Texture2D)btn_avanti, btn_indietro, btn_ritorna (Texture2D)rect_avanti, rect_indietro, rect_ritorna (Rectangle)pagina_corrente(int)total e_pagine (int)record_caricati (int)
StatoGioco	Enumerazione (enum)	Macchina a stati finiti globale che determina la scena corrente renderizzata nel ciclo del main.c.	STATO_MENU, STATO_ACCEDI, STATO_REGISTRATISTATO_MENU_SCELTA, STATO_GIOCO_1VS1STATO_ARCHIVIO, STATO_DEBUG
suono_bottone	Variabile Statica (static Sound)	Cache locale della risorsa audio per l'effetto di click d'interfaccia (allocata con pattern anti-leak).	Tipo: Sound (Raylib)
musica_corrente	Variabile Statica (static Music)	Stream del flusso audio di sottofondo in esecuzione ciclica continua (audio.c).	Tipo: Music (Raylib)
traccia_vittoria / traccia_sconfitta	Risorse Audio (Sound)	Effetti audio riprodotti alla fine del match in base all'esito (audio.c).	Tipo: Sound (Caricati da <i>win.mp3</i> / <i>sconfitta.mp3</i>)
foca_popup / popup_deltaplano	Texture Video (Texture2D)	Sprite grafici centralizzati usati per i popup delle animazioni degli eventi speciali (graphics.c).	Tipo: Texture2D
effettiMenuAttivi	Variabile Statica (static bool)	Interruttore software locale per mutare o abilitare i suoni di notifica dei menu.	Valori: true / false

codice_errore	Variabile Statica (static int)	Registro dell'indice di errore di validazione delle credenziali per i banner grafici (menu.c).	0: Nessun errore1: Utente non trovato2: Password errata3: Nickname in uso4: Spazio vietato
MAX_INPUT_STR	Costante (#define)	Limite massimo invalicabile di caratteri per le stringhe di input utente.	Valore: 10
BUFFER_AUTH_SIZE	Costante (#define)	Dimensione fisica degli array di stringhe inclusiva del carattere terminatore di stringa \0.	Valore: 12
SCHERMO_LARGHEZZA	Costante (#define)	Dimensione dell'asse X della finestra di rendering dell'applicazione (graphics.h).	Valore: 1200.0f
SCHERMO_ALTEZZA	Costante (#define)	Dimensione dell'asse Y della finestra di rendering dell'applicazione (graphics.h).	Valore: 800.0f

2.1 Effetti e suoni

GRAPHICS.H

- **Texture2D CaricaSicuro(const char *path);**
 - **Descrizione:** Questa funzione si occupa di caricare una texture in memoria a partire dal percorso (path) specificato. Include un controllo di sicurezza per verificare l'esistenza del file sul disco: se l'asset è presente viene caricato normalmente, altrimenti stampa un avviso nel terminale e restituisce una texture di fallback (un rettangolo di emergenza) per prevenire il crash improvviso dell'applicazione.
- **void InizializzaGrafica(Grafica *gfx);**
 - **Descrizione:** È la funzione di configurazione iniziale del modulo grafico. Riceve il puntatore alla struttura Grafica e alloca in memoria video (VRAM) tutti gli elementi visivi necessari alla partita: l'immagine di sfondo, le immagini della foca e del deltaplano per le animazioni, le schermate di vittoria/sconfitta, il retro delle carte e carica ciclicamente la matrice tridimensionale contenente le texture di tutte le singole carte da gioco.
- **void DisegnaPartita(Grafica *gfx, Partita *p);**
 - **Descrizione:** Rappresenta il motore di rendering principale del tavolo da gioco durante il match. Leggendo lo stato logico corrente della struttura Partita, si occupa di disegnare a schermo lo sfondo, il mazzo di pesca, la carta in cima agli scarti, le carte inclinate ad arco nella mano del giocatore umano, i dorsi delle carte nascoste del bot, le animazioni di spostamento delle carte, i messaggi pop-up e le interfacce per la scelta del colore o di fine partita.

- **void ScaricaGrafica(Grafica *gfx);**
 - **Descrizione:** Questa funzione si occupa della pulizia e della corretta gestione della memoria alla chiusura del gioco o del modulo. Esegue la deallocazione (unload) sistematica di tutte le texture precedentemente caricate nella struttura Grafica (sfondi, pulsanti, animazioni e l'intero set di carte), liberando la memoria video ed evitando pericolosi leak di memoria (*memory leaks*).
- **void DisegnaTestoPixelGrassetto(const char* testo, int x, int y, int fontSize, Color baseColor);**
 - **Descrizione:** È una funzione di utilità adibita al rendering del testo a schermo. È progettata per simulare graficamente un carattere in grassetto in stile retro pixel art. Per farlo, applica un algoritmo che disegna il testo più volte in colore nero con piccoli scostamenti direzionali (creando un contorno o un'ombreggiatura spessa ad alto contrasto) prima di sovrapporre il testo finale nel colore (baseColor) e nella dimensione (fontSize) scelti alle coordinate x e y.

2.1.1 GUI

- **Effetti**



- **Carte**

-Carte dallo 0-9



-Carte azione



-Carte speciali



-Retro carta



- **Menù**

Con Registrazione Utente

Senza Registrazione Utente



2.2 Gestione Asset

Le texture sono caricate tramite percorsi relativi, per esempio:

- Carte/rosso_0.png;
- Sfondo/sfondo_partita.png;
- Pulsanti/sospendi.png;
- Animazioni/win.png;
- Musiche/musica.mp3.

Per aggiungere una nuova carta o sostituire una grafica, mantenere i nomi attesi dal codice. Le carte sono caricate con convenzioni come:

- Carte/rosso_5.png;
- Carte/blu_stop.png;
- Carte/giallo_rev.png;
- Carte/verde_p2.png;
- Carte/wild.png;
- Carte/wild_4.png.

2.2.1 Note di manutenzione

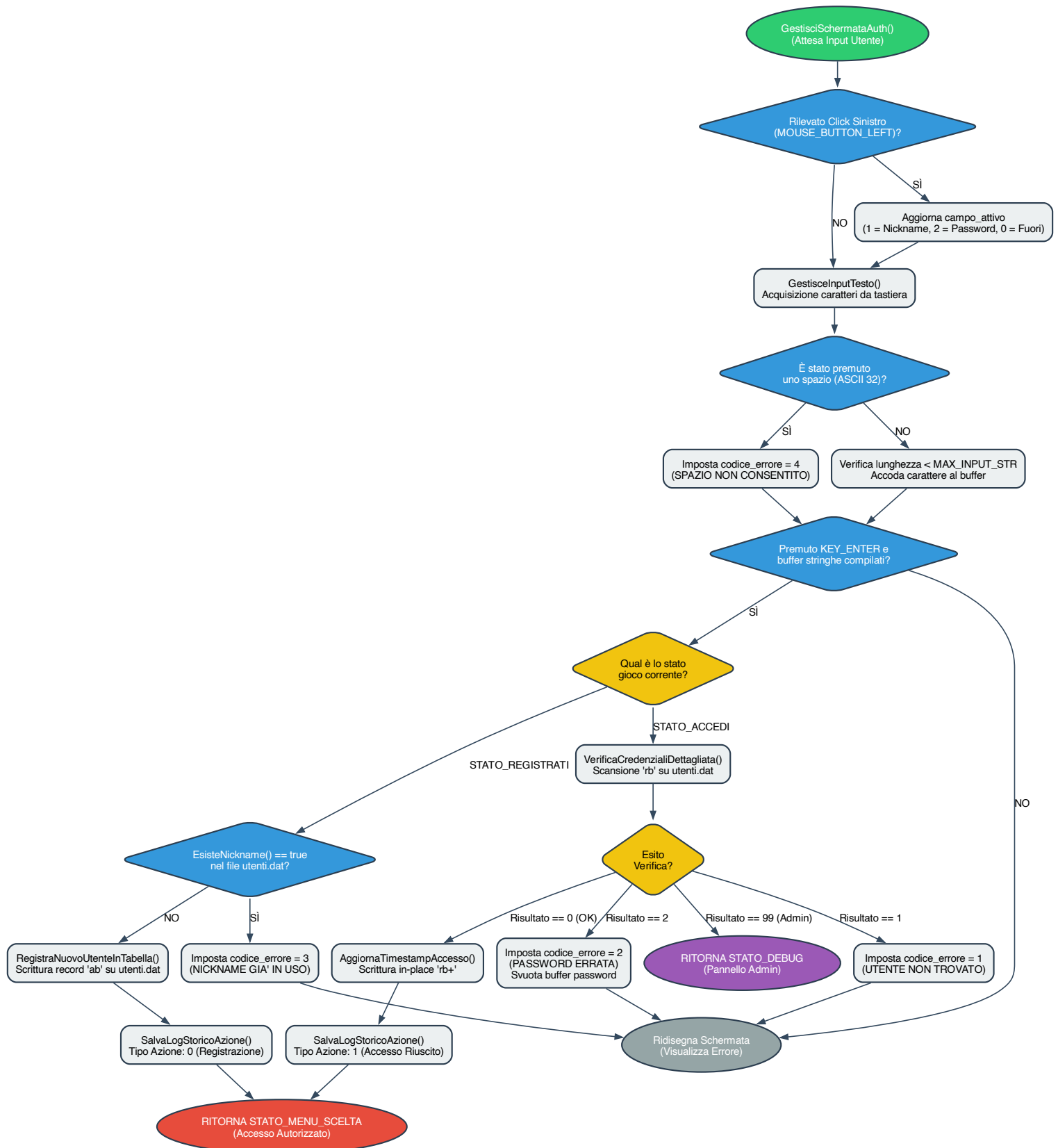
- Le dimensioni finestra, carta e mano massima sono definite in GAME.H .
- Le strutture vengono salvate direttamente su file binari: modificare l'ordine o la dimensione dei campi puo rendere incompatibili i vecchi salvataggi.
- Le password sono salvate in chiaro nei file locali: il sistema e adatto a un progetto didattico, non a un uso reale.
- Il rendering e legato alla risoluzione 1200x800; per supportare risoluzioni diverse serve rendere proporzionali coordinate e layout.
- Alcune funzioni globali e variabili condivise sono usate tra MAIN.C e ARCHIVIO.C per pulsanti e sfondi; eventuali refactor dovrebbero ridurre queste dipendenze.
- GAME.C contiene anche una funzione SalvaPartita(...) che salva su salvataggi_partite.dat, mentre il flusso principale usa SalvaPartitaSospesa(...) da ARCHIVIO.C .

Flusso tipico di gioco

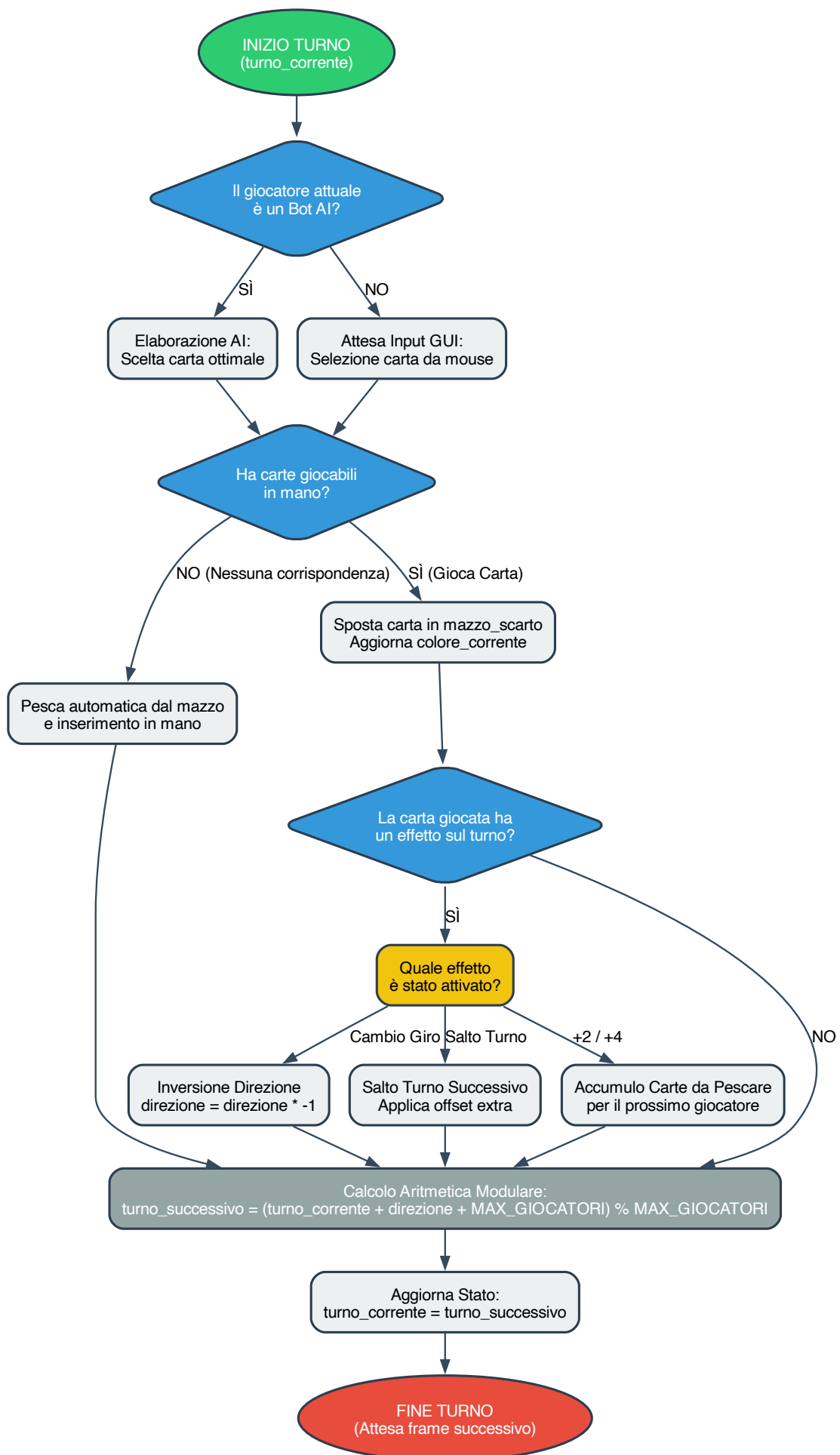
1. L'utente apre il programma.
2. Sceglie login, registrazione o ospite.
3. Dal menu scelta avvia 1vs1 o multigiocatore.
4. Durante la partita:
 - clicca una carta valida per giocarla;
 - clicca il mazzo per pescare;
 - se la carta pescata e giocabile può passare cliccando di nuovo il mazzo;
 - se gioca una carta nera sceglie il nuovo colore;
 - se è registrato può sospendere la partita.
5. A fine partita il programma aggiorna vittorie o sconfitte per l'utente registrato.
6. Dall'archivio l'utente può riprendere una partita salvata.

2.3 Flowchart gestione gioco

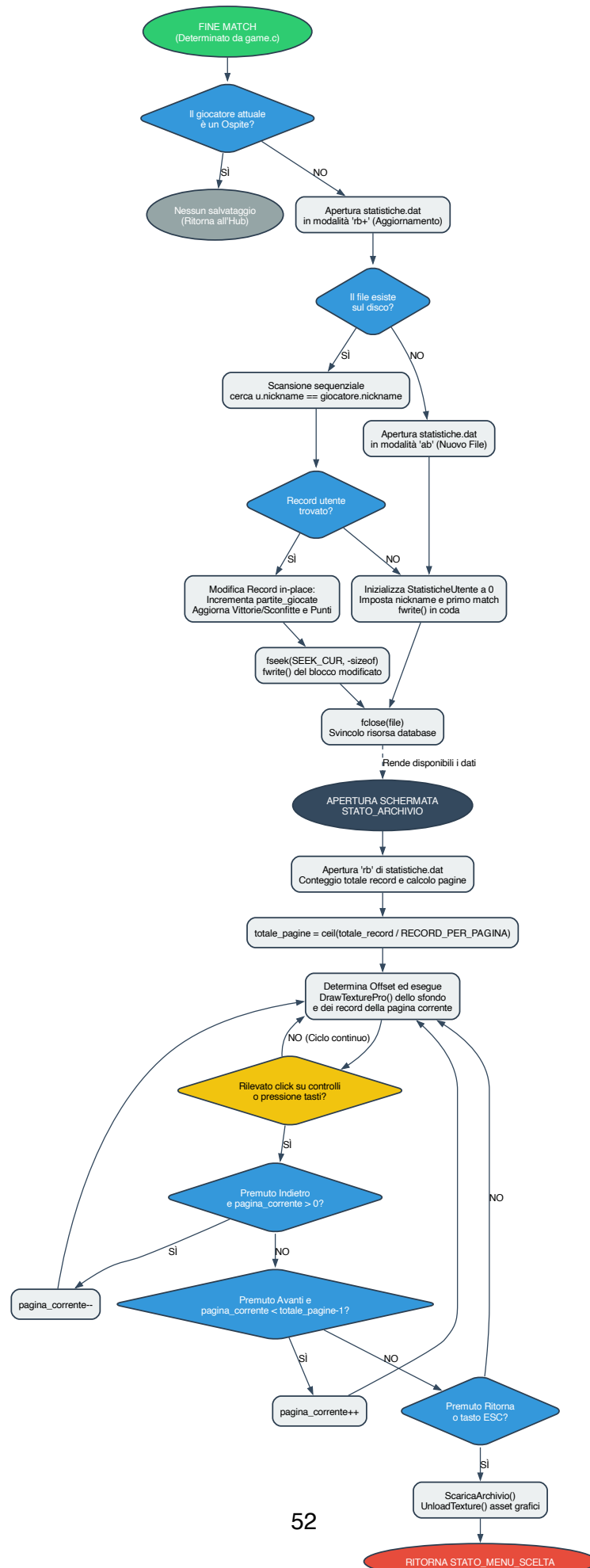
2.3.1 Gestione login e registrazione



2.3.2 Gestione logica di gioco

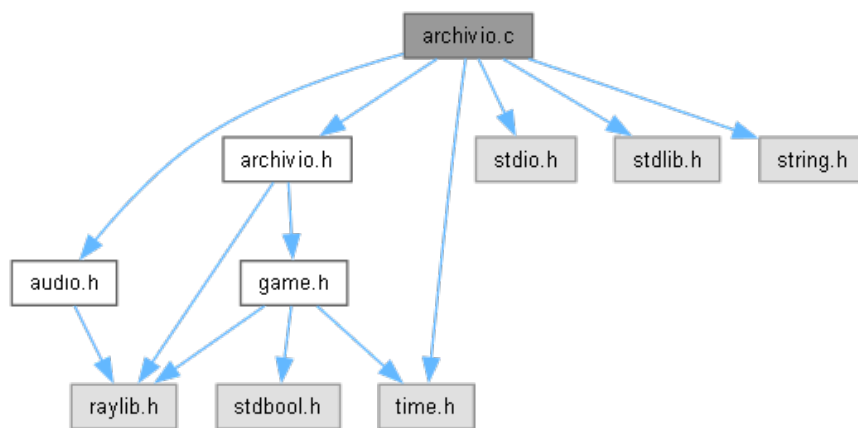


2.3.3 Gestione archivio e statistiche

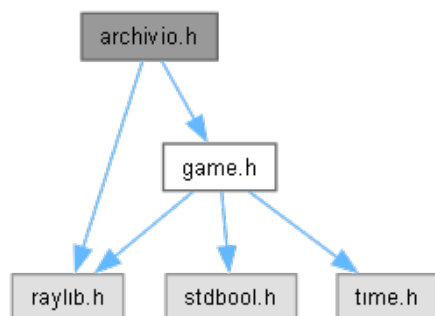


2.4 Grafì delle dipendenze

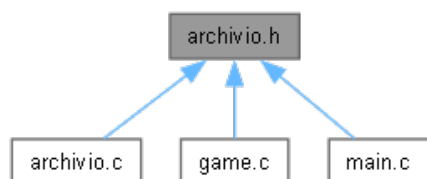
- **archivio.c**



- **archivio.h**



File che includono direttamente o indirettamente **archivio.h**



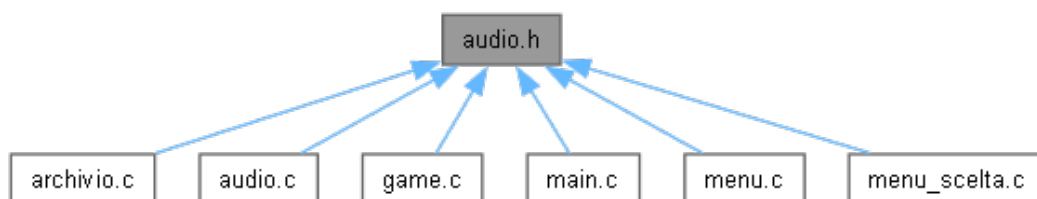
- **audio.c**



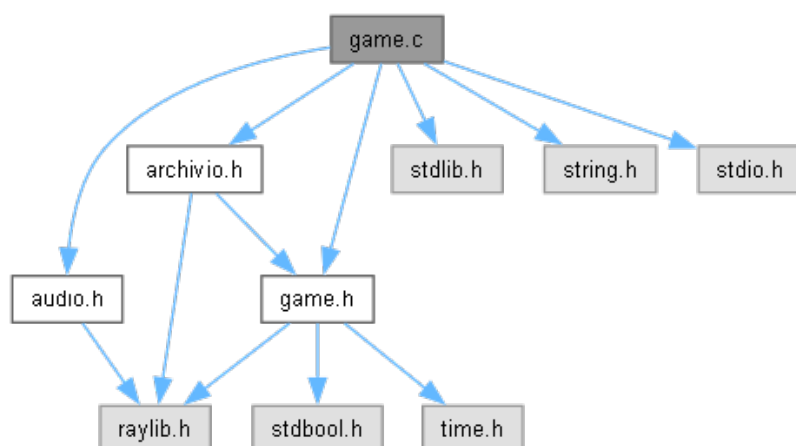
- **audio.h**



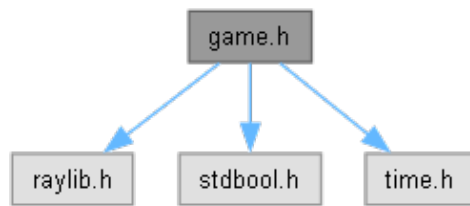
File che includono direttamente o indirettamente audio.h



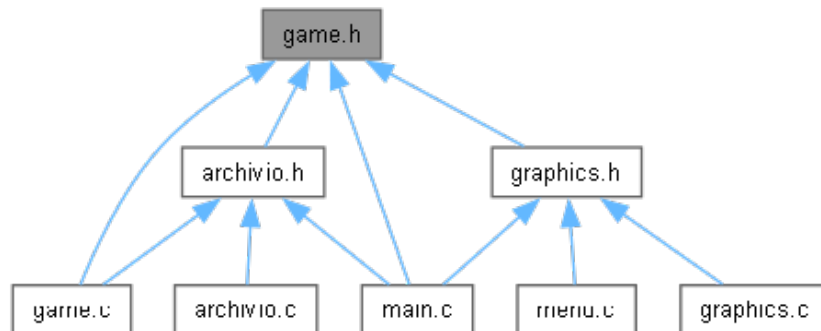
- **game.c**



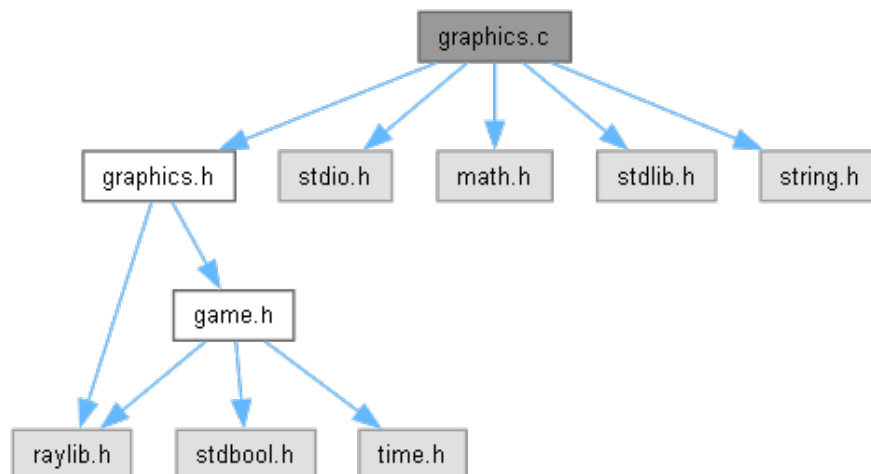
- **game.h**



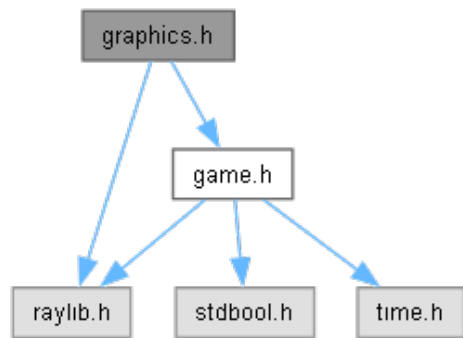
File che includono direttamente o indirettamente `game.h`



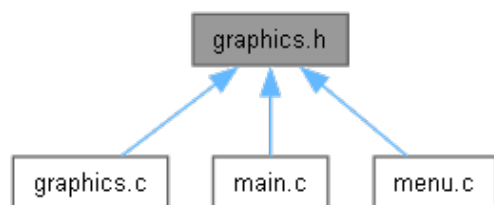
- **graphics.c**



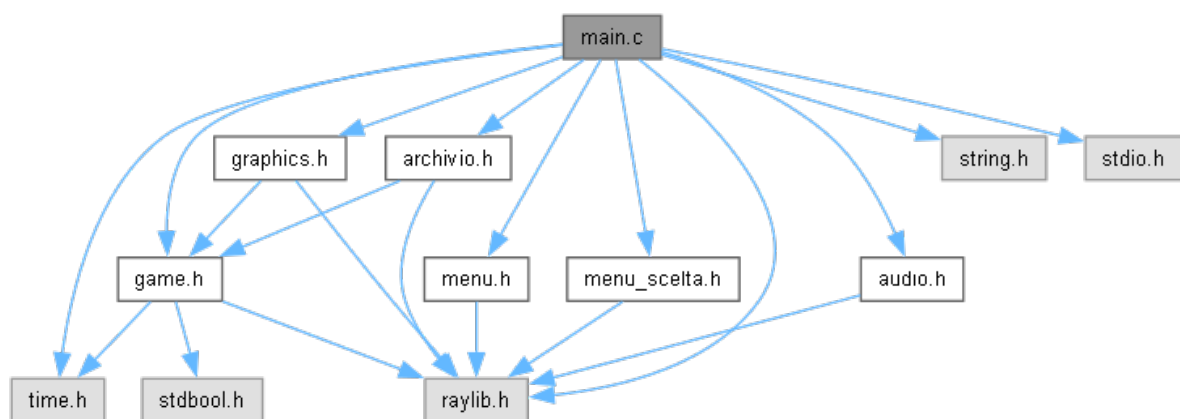
- **graphics.h**



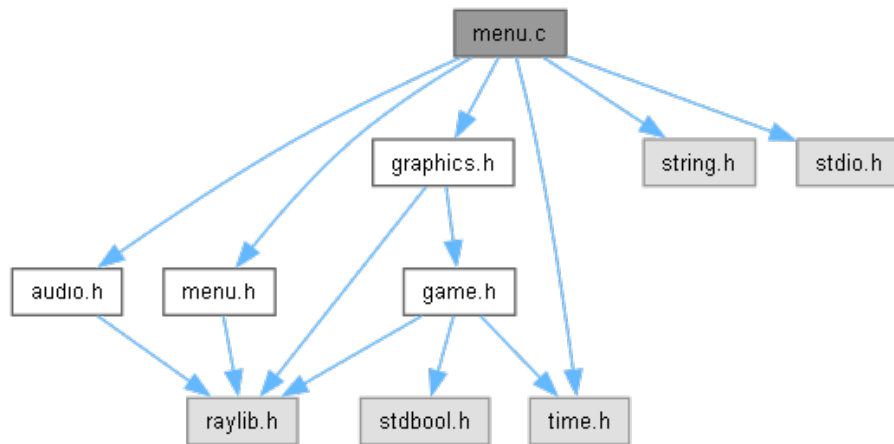
File che includono direttamente o indirettamente **graphics.h**



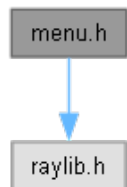
- **main.c**



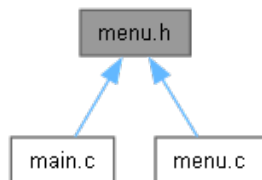
- **menu.c**



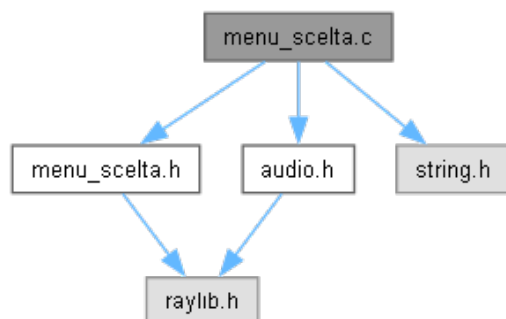
- **menu.h**



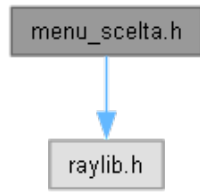
File che includono direttamente o indirettamente `menu.h`



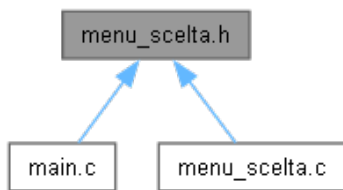
- **menu_scelta.c**



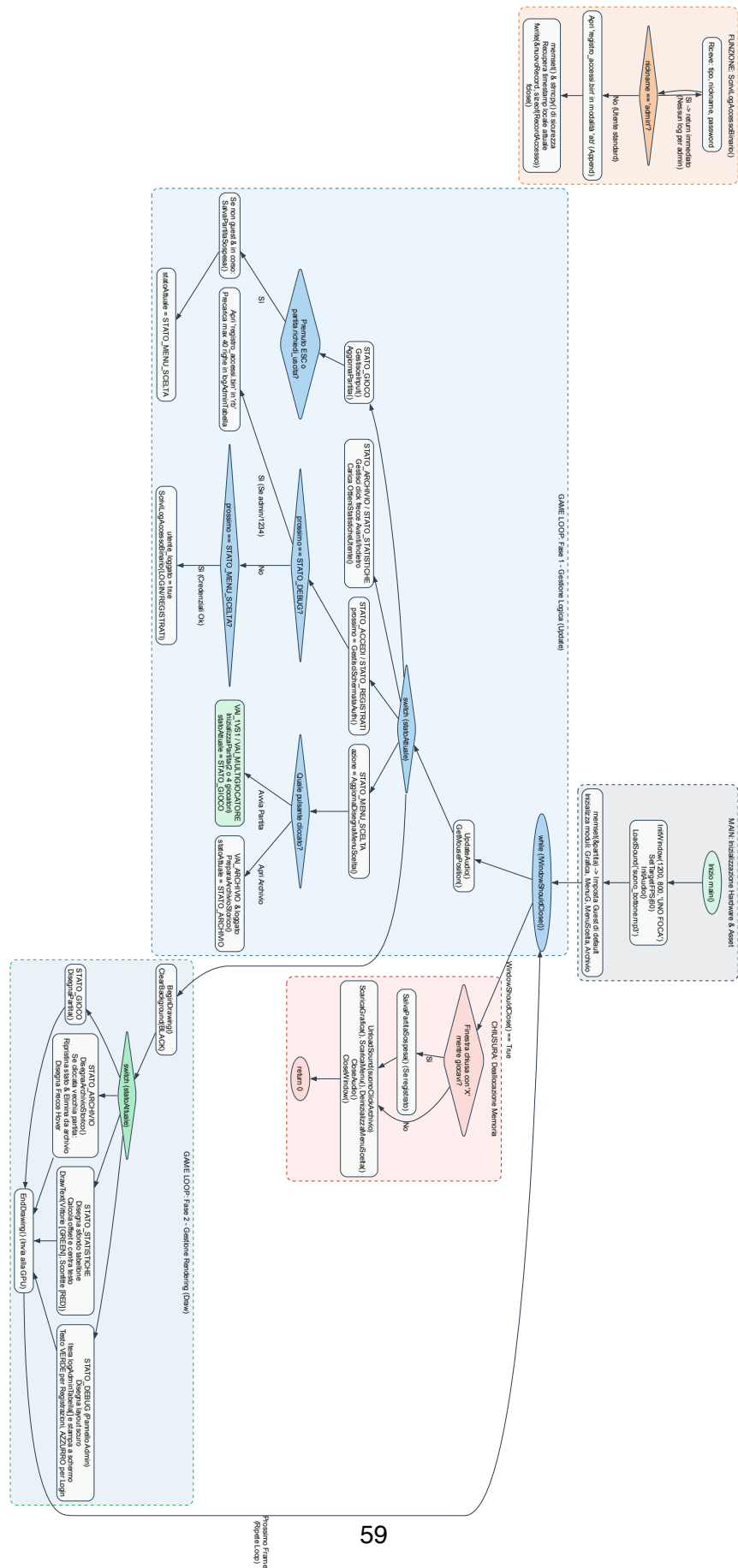
- **menu_scelta.h**



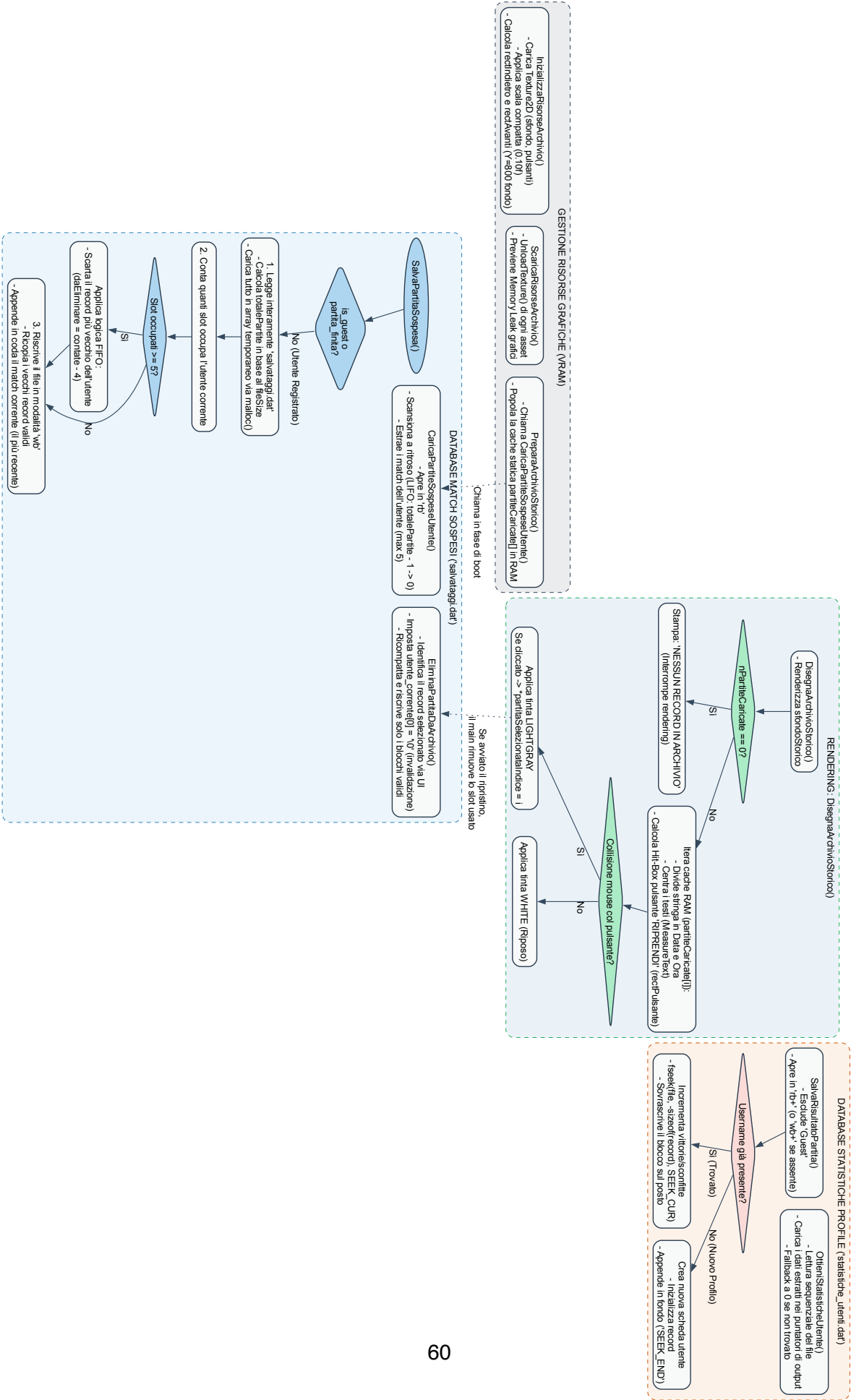
File che includono direttamente o indirettamente menu_scelta.h



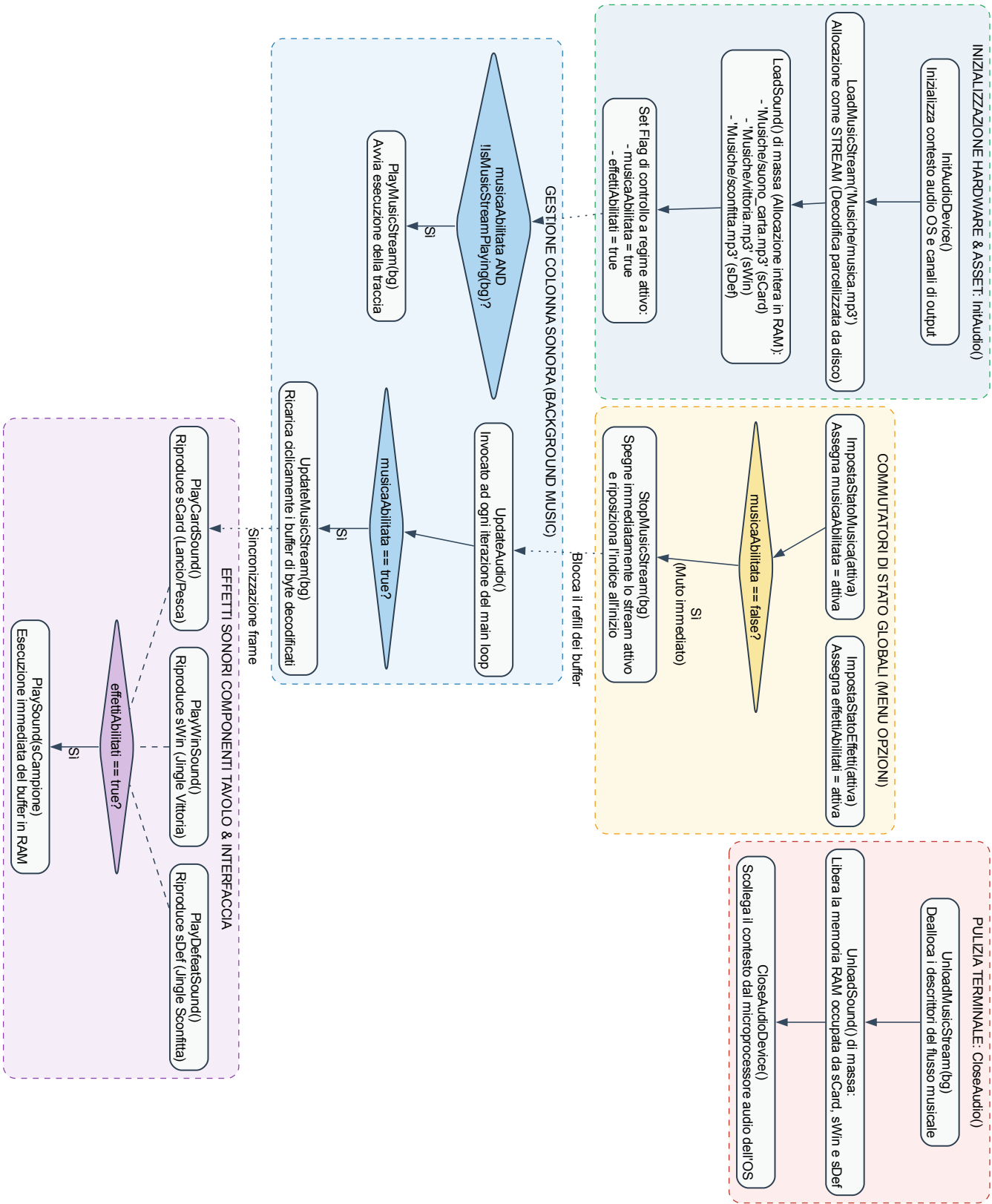
2.5.1 Main.c



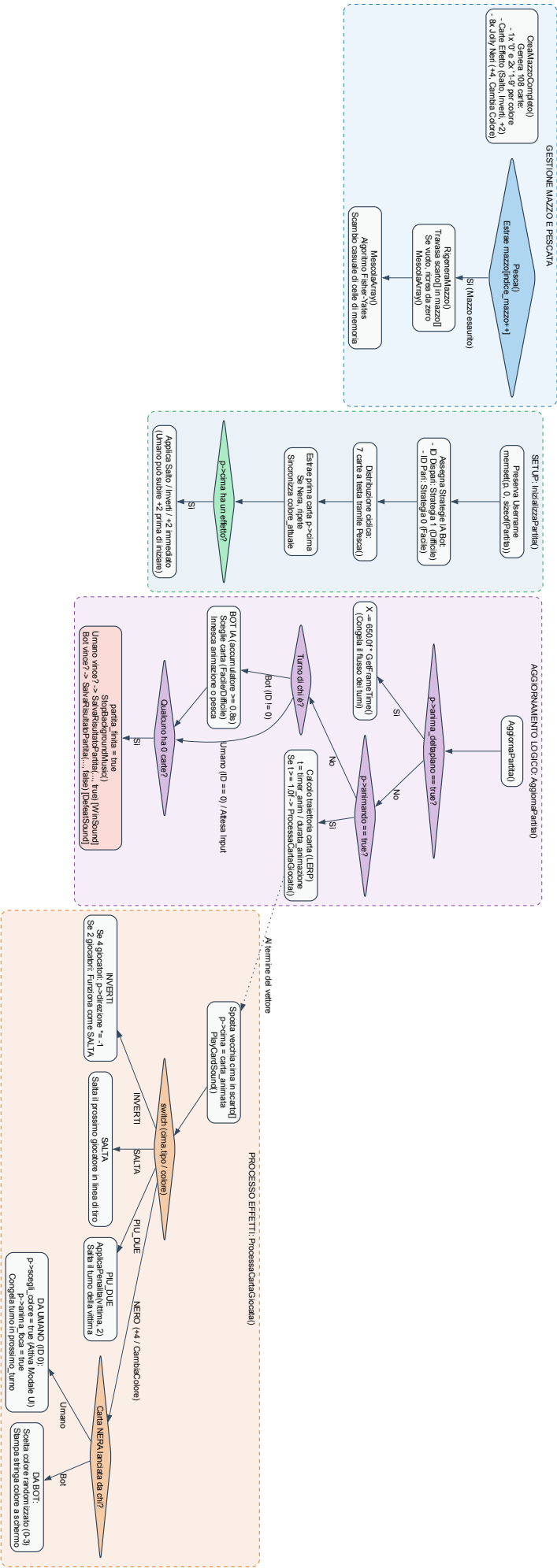
2.5.2 Archivio.c



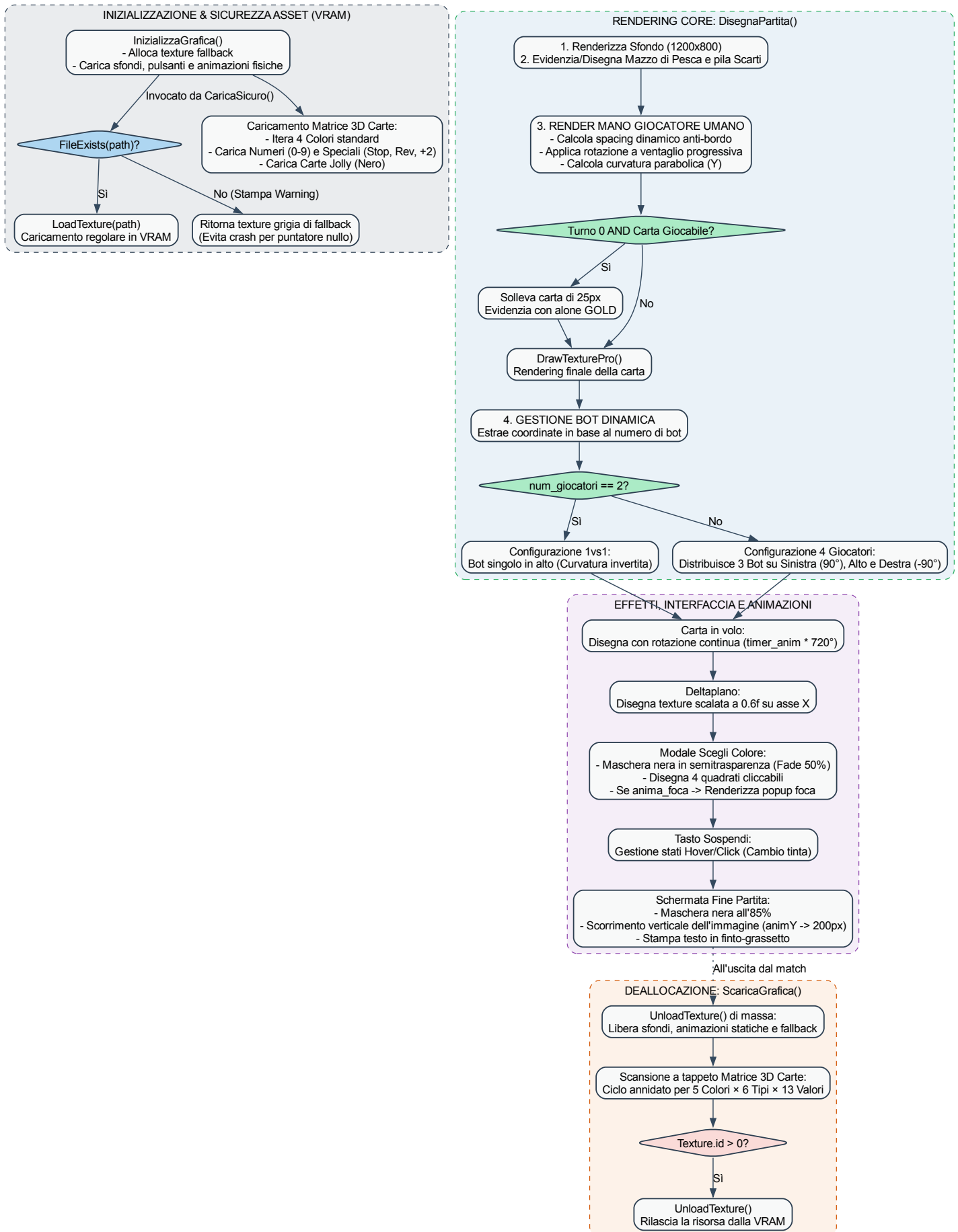
2.5.3 Audio.c



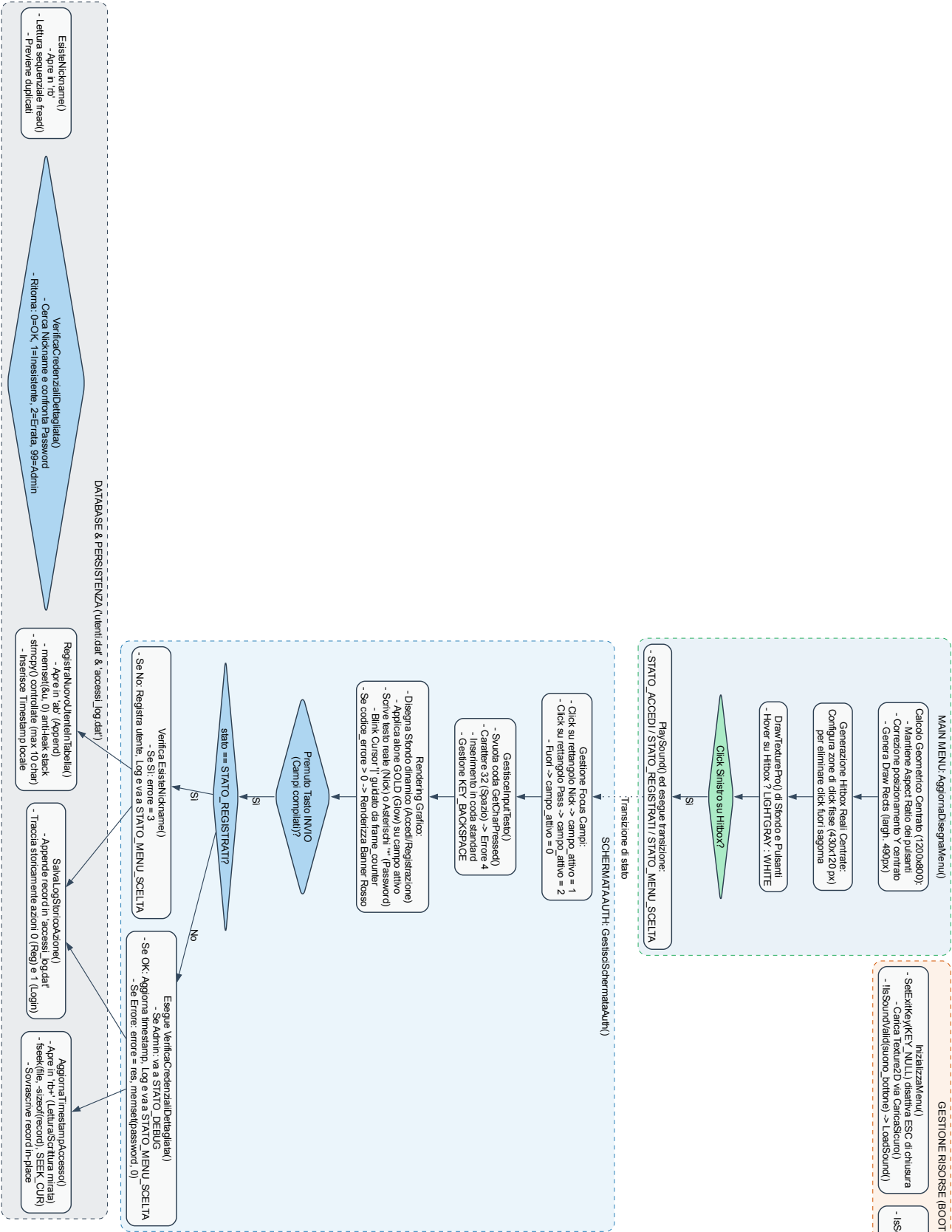
2.5.4 Game.c



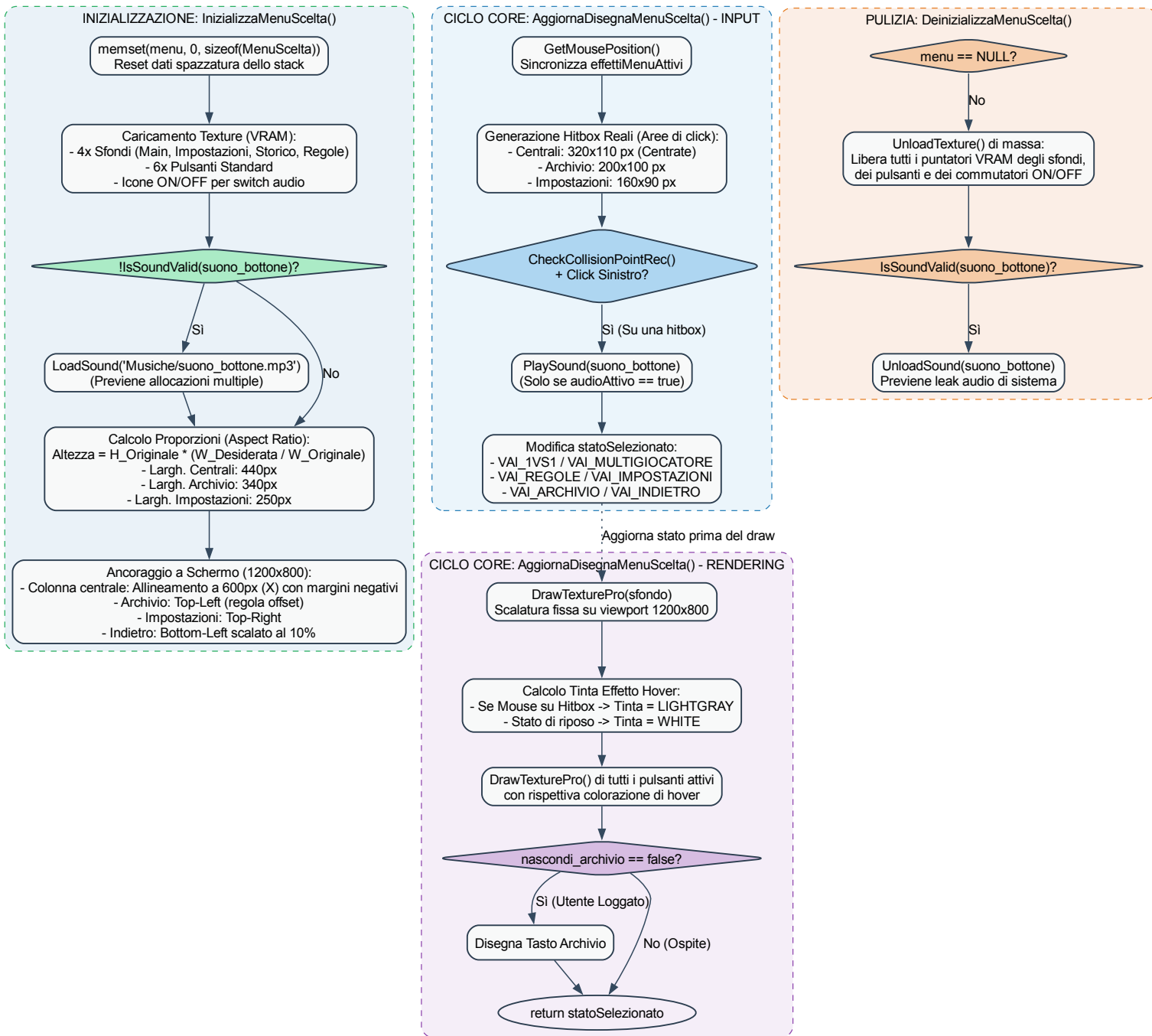
2.5.4 Graphics.c



2.5.5 Menu.c



2.5.6 Menu_scelta.c



3) Codifica

3.0 Stati del MENU

Flusso generale dell'applicazione

Il file **MAIN.C** inizializza finestra, audio, grafica, menu e archivio. Poi entra nel ciclo principale Raylib:

1. aggiorna l'audio con **UpdateAudio()**;
2. legge mouse e tastiera;
3. aggiorna la schermata corrente tramite lo stato **StatoGioco**;
4. disegna la schermata con **BeginDrawing()** e **EndDrawing()**;
5. alla chiusura scarica texture, suoni e dispositivo audio.

Gli stati principali sono definiti in **MENU.H** :

Stato di Partenza	Evento	Stato di Arrivo	Dettagli Tecnici
[START APPLICAZIONE]	Inizializzazione applicazione.	STATO_MENU	Il main imposta lo stato iniziale a STATO_MENU.
STATO_MENU	Click "Gioca come ospite" / "Inizia" (B).	STATO_GIOCO	Avvia la partita diretta senza registrazione.
STATO_MENU	Click "Accedi" (A).	STATO_ACCEDI	Carica le risorse della UI di login.
STATO_MENU	Click "Registrati".	STATO_REGISTRATI	Carica le risorse della UI di registrazione.
STATO_ACCEDI	Login OK (C).	STATO_GIOCO	Credenziali utente standard corrette.
STATO_ACCEDI	Errore Password / Nick (D).	STATO_ACCEDI	Resta nello stato, visualizza un avviso di errore a schermo.
STATO_ACCEDI	Login Speciale (I).	STATO_DEBUG	Inserimento credenziali ADMIN ("ADMIN"/"1234").
STATO_ACCEDI	Click "Indietro" (C).	STATO_MENU	Annulla l'operazione di login e scarica le risorse UI dedicate.

Stato di Partenza	Evento	Stato di Arrivo	Dettagli Tecnici
STATO_REGISTRA TI	Registrazione OK.	STATO_MENU	Registrazione completata con successo nel file locale. Scarica risorse e torna al menu.
STATO_REGISTRA TI	Errore Input / Nick Usato.	STATO_REGISTR ATI	Resta nello stato e visualizza l'errore (speculare a login fail).
STATO_REGISTRA TI	Click "Indietro".	STATO_MENU	Annulla la registrazione (speculare a login back).
STATO_GIOCO	Vittoria o Sconfitta / Punteggio Raggiunto (E).	STATO_MENU	Stato Finale della Partita. Scarica le risorse della partita e salva le statistiche locali prima di tornare al menu.
STATO_DEBUG	Click "Indietro" / "Esci Debug" (J).	STATO_MENU	Torna al menu principale.
QUALSIASI STATO	Chiudi Finestra (K).	[FINE]	L'evento WindowShouldClose() nel ciclo main scarica tutte le risorse globali e chiude il gioco.

L'automa è strutturato su un unico processo principale che parte dallo stato iniziale STATO_MENU e può terminare solo con l'evento **WindowShouldClose()** da qualsiasi stato.

3.1 GAME.C

- In GAME.c
- **OttieniPosizioneMazzoGiocatore:** Restituisce le coordinate geometriche (Vector2) sullo schermo per il posizionamento grafico delle carte di un giocatore, differenziando la posizione tra il giocatore umano (in basso) e i bot in base alla modalità di gioco selezionata (1vs1 con bot in alto, oppure 1vs3 con bot a sinistra, in alto e a destra).
- **MescolaArray (funzione statica):** Algoritmo di mescolamento che randomizza l'ordine degli elementi di un array di carte. Sfrutta la funzione GetRandomValue di Raylib per scambiare casualmente di posto le carte nel mazzo.
- **CreaMazzoCompleto (funzione statica):** Genera e popola l'array del mazzo con le 108 carte regolamentari del gioco UNO: inserisce le carte numeriche (da 0 a 9), le carte speciali colorate (Salto Turno, Inverti Giro, +2) per i 4 colori base e infine le carte speciali nere (Cambio Colore e +4). Restituisce il numero totale di carte create.
- **RigeneraMazzo (funzione statica):** Ricostituisce il mazzo di pesca quando le carte finiscono. Se ci sono carte nella pila degli scarti, le sposta nel mazzo principale e le rimescola (lasciando intatta la carta in cima); in caso contrario, rigenera da zero un mazzo completo di sicurezza.
- **Pesca:** Estrae la carta successiva in cima al mazzo di pesca e fa avanzare l'indice di lettura. Se rileva che il mazzo è esaurito, invoca automaticamente RigeneraMazzo.

- **ImpostaMessaggio (funzione statica):** Configura una stringa di testo da visualizzare nell'interfaccia grafica (es. avvisi sulle mosse dei bot) assicurando la corretta terminazione della stringa e inizializza un timer di visibilità a schermo impostato a 3.5 secondi.
- **ApplicaPenalita (funzione statica):** costringe un determinato giocatore a prelevare dal mazzo un numero specifico di carte a causa degli effetti degli attacchi subiti (come i nodi +2 o +4), verificando di non superare il limite massimo consentito in mano (MANO_MAX).
- **PuoGiocare:** verifica i regolamenti ufficiali di UNO per stabilire se una determinata carta in mano può essere legalmente giocata sopra quella che si trova in cima alla pila degli scarti, controllando la corrispondenza di colore, valore numerico o tipo (oppure se si tratta di una carta nera).
- **GiocatoreHaMosse:** Scansiona l'intera mano di uno specifico giocatore (umano o bot) e restituisce un valore booleano che indica se il soggetto ha o meno almeno una mossa valida da poter giocare nel turno corrente.
- **InizializzaPartita:** predispone una nuova sessione di gioco azzerando la struttura dati principale. Configura il numero di partecipanti, assegna in modo alternato le strategie di difficoltà IA ai bot (Facile o Difficile), crea e mescola il mazzo, distribuisce 7 carte a testa, stabilisce la prima carta sul tavolo e ne applica gli eventuali effetti iniziali (es. sanzioni immediate di cambio turno o +2).
- **SalvaPartita:** se l'utente è loggato (non ospite) e il match non è concluso, esporta l'intera struttura di gioco scrivendola in modalità binaria in coda al file locale salvataggi_partite.dat, memorizzando

anche il nickname corrente e un timestamp con data e ora di sistema.

- **ProcessaCartaGiocata (*funzione statica*):** gestisce l'applicazione degli effetti regolamentari della carta appena posizionata sul tavolo: sposta la vecchia cima negli scarti, riproduce il suono della carta, calcola il turno del giocatore successivo applicando le penalità delle carte speciali (+2, +4, Salto Turno, Inversione della direzione di giro) e attiva i flag per la selezione del colore se è stata giocata una carta nera.
- **GestisceInput:** Intercetta i comandi del giocatore umano. Gestisce la richiesta di uscita via tasto ESC o tramite il pulsante "Sospendi", la selezione dei colori dal pop-up e coordina la logica di gioco nel turno 0: permette la pescata singola dal mazzo, lo sblocco del passaggio del turno al secondo clic (se non si hanno mosse) e la selezione/animazione della carta da calare sul tavolo dalla propria mano.
- **SceltaStrategica (*funzione statica*) :** algoritmo decisionale per l'IA dei bot con difficoltà difficile. Implementa una priorità strategica: scansiona la mano per giocare prima le normali carte numeriche valide in modo da conservare le carte speciali per i momenti critici; se non ha numeri giocabili, opta per una delle sue carte speciali.
- **AggiornaPartita:** lo scorrimento del deltaplano per le notifiche di "UNO", i timer dei messaggi e l'interpolazione fluida delle coordinate delle carte durante le animazioni. Nel turno dei bot, introduce un ritardo artificiale di 0.8 secondi ed esegue le loro mosse (pescata automatica o calata della carta tramite IA facile/ SceltaStrategica). Infine, controlla le condizioni di fine partita registrando la vittoria/sconfitta nel modulo archivio.

3.2 MENU.C

- In MENU.c

Il modulo menu.c è responsabile della gestione completa dell'interfaccia utente iniziale, dei cicli di interazione pre-partita e dei sottosistemi di autenticazione e registrazione locale. Il modulo funge da coordinatore degli stati primari dell'applicazione (STATO_MENU, STATO_ACCEDI, STATO_REGISTRATI), interfacciandosi direttamente con il modulo grafico e i flussi di memorizzazione persistente su file binari.

3.2.1 Testo ed effetti

Questa sezione analizza l'architettura logica adibita alla cattura dell'input da tastiera, al filtraggio dei caratteri e ai feedback visivi dinamici distribuiti all'utente durante la navigazione e la digitazione.

- **Gestione e buffering dell'input testuale:** attraverso la funzione `GestisceInputTesto`, il sistema implementa un ciclo di cattura asincrono basato sulle primitive Raylib `GetCharPressed()` e `IsKeyPressed(KEY_BACKSPACE)`.
L'input viene validato in tempo reale applicando i seguenti vincoli hardware e logici:
 - *Limitazione di memoria:* La lunghezza massima delle stringhe (nickname e password) è rigidamente bloccata a un massimo di 10 caratteri per prevenire alterazioni strategiche o anomalie di visualizzazione nei form dell'interfaccia.
 - *Filtraggio dei caratteri:* Nel campo password (`id == 2`), viene intercettato il carattere di spazio (codice ASCII 32); in caso di inserimento, viene sollevato un flag di errore (`codice_errore=4`) bloccando la stringa.

- **Mascheramento dei caratteri password:** per garantire la riservatezza all'interno della schermata di autenticazione, il modulo esegue una conversione procedurale a runtime.
La stringa reale della password viene letta in memoria e stampata a schermo convertendo graficamente ogni singolo carattere inserito in un asterisco (*) mediante un array secondario.
- **Cursore lampeggiante procedurale:** l'effetto del cursore di digitazione è gestito in modo deterministico attraverso la variabile incrementale `frame_counter`. Sfruttando un'operazione di modulo applicata alla frequenza di aggiornamento grafico ($(mg - > frame_counter / 20) \% 2 == 0$), il cursore viene renderizzato alternativamente ad ogni frame attivo solo all'interno della casella di testo attualmente selezionata (`campo_attivo`).
- **Effetti Visivi e Highlight Interattivi:**
 - *Glow Effetto:* Quando un campo di input (Nickname o Password) ottiene il focus tramite collisione del mouse, viene calcolato un rettangolo espanso (`rGlow`) con uno spessore aggiuntivo di 5 pixel. Questo rettangolo viene renderizzato utilizzando la texture del campo combinata con la tinta GOLD, creando un effetto visivo di retroilluminazione ad alta fedeltà.
 - All'interno della funzione `AggiornaDisegnaMenu`, la posizione del cursore viene costantemente monitorata rispetto alle hitbox dei pulsanti tramite `CheckCollisionPointRec`. Al verificarsi della collisione (`hovering`), il colore di rendering del pulsante viene alterato applicando la maschera `LIGHTGRAY`, garantendo un feedback visivo immediato prima del clic.

3.2.2 Data e ora

Il modulo integra la libreria di sistema standard <time.h> per gestire la temporizzazione degli eventi di accesso dei giocatori all'interno del sistema di identificazione locale.

- **Acquisizione del tempo di sistema:** Ogni volta che viene completata con successo un'operazione di registrazione o di login, il sistema interroga l'orologio hardware della macchina ospite invocando la funzione `time(NULL)`.
- **Formattazione e localizzazione:** Il timestamp grezzo ottenuto (`time_t`) viene convertito in una struttura di tempo locale scomposta (`struct tm`) mediante la funzione `localtime`. Successivamente, la stringa temporale viene formattata in modo sicuro tramite la routine `strftime`, che popola l'array dedicato (`ultimo_accesso` o `data_ora`) rispettando rigorosamente il formato standard europeo a 24 ore: `GG/MM/AAAA HH:MM:SS` (es. `29/05/2026 14:00:00`). Questa stringa standardizzata viene poi salvata nel file per tracciare storicamente l'attività degli utenti.

3.2.3 Salvataggi e Persistenza Dati

Il file implementa una gestione della persistenza locale a basso livello basata su flussi di file binari strutturati (`FILE *`), garantendo velocità di esecuzione e totale indipendenza da database o server esterni. Il modulo gestisce nello specifico due diversi file binari:

1. **Anagrafe Utenti (`utenti.dat`):** Memorizza le informazioni strutturate di tipo `UtenteRegistrato` (comprendenti nickname, password, stringa dell'ultimo accesso e flag dell'ultima azione eseguita). Il modulo effettua due tipi di accesso ottimizzato a questo file:
 - *Scrittura in coda ("ab" - Append Binary):* Utilizzata nella funzione `RegistraNuovoUtenteInTabella` all'atto della creazione di un nuovo profilo. Per mitigare i rischi di vulnerabilità legati alle stringhe non terminate in C, il sistema esegue preventivamente

un memset di azzeramento della struct e applica una copia protetta con strncpy, forzando esplicitamente il carattere terminatore nullo (\0) all'undicesimo byte dell'array (u.nickname[10] = '\0').

- *Aggiornamento in Place ("rb+" - Read/Write Binary)*: Utilizzato nella funzione `AggiornaTimestampAccesso`. Invece di riscrivere interamente il file o duplicare le righe, il sistema esegue una scansione sequenziale con `fread`. Una volta individuato il record corrispondente al nickname dell'utente loggato, la funzione aggiorna i dati in memoria, riposiziona con precisione il cursore del file all'inizio del record corrente mediante la funzione di posizionamento `fseek(file, -((long)sizeof(UtenteRegistrato)), SEEK_CUR)` ed esegue una sovrascrittura mirata del singolo blocco tramite `fwrite`.

2. **Registro Storico degli Accessi (accessi_log.dat)**: Gestito in modalità `append ("ab")`, questo file mantiene una traccia storica immutabile e separata di ogni singola transazione effettuata nel modulo di autenticazione. Ogni record memorizzato (`LogAccesso`) mappa esplicitamente le credenziali inserite, il timestamp dell'evento e il tipo di operazione (0 per la registrazione, 1 per il login effettuato), fornendo all'applicazione una base dati strutturata consultabile in seguito dal pannello di controllo dell'amministratore.

3.3 MENU_SCELTA.C

- In MENU_SCELTA.c

Il modulo menu_scelta.c implementa l'interfaccia logica e grafica di secondo livello dell'applicazione, fungendo da smistatore centrale degli stati del software. Il suo compito principale è catturare le intenzioni del giocatore e indirizzare il ciclo del main.c verso i diversi contesti funzionali: la partita locale (1vs1), il comparto multigiocatore, il pannello delle regole, la schermata delle impostazioni audio o l'archivio storico delle partite sospese.

- **InizializzaMenuScelta:** questa funzione si occupa dell'azzeramento iniziale della struttura dati e del caricamento di tutti gli asset multimediali necessari al menu. Carica dal disco le immagini di sfondo per le sezioni principali, le impostazioni, l'archivio e le regole, insieme alle texture di tutti i pulsanti interattivi e al file audio per l'effetto sonoro del clic. Inoltre, imposta i flag di attivazione predefiniti per l'audio e calcola matematicamente le coordinate geometriche, i margini e le dimensioni di ogni pulsante, posizionandoli in modo simmetrico e centrato rispetto alla risoluzione nativa dello schermo.
- **AggiornaDisegnaMenuScelta:** Rappresenta la routine fondamentale del ciclo di esecuzione del menu. Gestisce a ogni frame sia la logica di interazione che il rendering visivo: rileva la posizione del mouse e calcola a runtime delle hitbox geometriche personalizzate (ridotte rispetto alle texture visive) per ottimizzare la precisione dei clic ed evitare selezioni involontarie. Gestisce l'effetto visivo di *hover* (illuminazione in grigio chiaro quando il mouse passa sopra un tasto) e riproduce l'effetto sonoro del pulsante solo se l'audio è abilitato nelle opzioni. Infine, applica un controllo di sicurezza condizionale: se l'utente sta giocando come ospite

(nascondi_archivio), la funzione occulta graficamente il pulsante dell'archivio storico e ne disabilita completamente le collisioni logiche per impedirne l'accesso. Al termine, restituisce lo stato o l'azione selezionata dal giocatore.

- **DeinializzaMenuScelta:** è la funzione deputata alla corretta pulizia e disallocazione delle risorse al cambio di contesto o alla chiusura dell'applicazione. Si occupa di scaricare tutte le texture dei quattro sfondi e di tutti i pulsanti caricati, e rilascia dalla memoria audio il campione sonoro del bottone, garantendo l'assenza di leak di memoria .

3.4 GRAPHICS.C

- In GRAPHICS.c

Il modulo GRAPHICS.C si occupa della pipeline grafica dell'applicazione, sfruttando le primitive di Raylib per mappare gli asset e gestire l'interfaccia visiva a una risoluzione fissa di 1200×800 pixel.

3.4.1 Pipeline di Rendering e Modularità dello Stato

Nelle ultime versioni del software, le funzioni fondamentali `BeginDrawing()` e `EndDrawing()` di Raylib sono state rimosse dall'interno di `DisegnaPartita` e riallocate direttamente nel loop principale del file `MAIN.C`. Questo refactoring architetturale garantisce un'elevata modularità, consentendo al main di gestire liberamente il rendering di qualsiasi schermata (Menu, Autenticazione, Gioco, Tabella Debug amministratore) all'interno dello stesso frame grafico.

3.4.2 Caricamento Sicuro e Gestione delle Risorse

- `CaricaSicuro(const char *path)`: Agisce come un wrapper di protezione per il caricamento delle texture. Prima di allocare l'immagine in VRAM, verifica l'effettiva presenza fisica del file su disco tramite `FileExists`. Se l'asset è mancante o corrotto, la funzione stampa un warning nel terminale e invoca `CreaFallback`, generando programmaticamente in memoria una texture geometrica grigia di emergenza per evitare il crash immediato dell'applicazione.
- `ScaricaGrafica(Grafica *gfx)`: Previene i memory leak sul sottosistema video deallocando sistematicamente dalla VRAM lo sfondo, i bottoni, le schermate di pop-up e l'intera matrice tridimensionale delle texture delle carte al cambio di contesto o alla chiusura del gioco.

3.4.3 Feedback Visivi e Animazioni UI

- **Evidenziazione Carte e Suggerimenti:** La funzione `EvidenziaCarta` disegna un rettangolo espanso dietro la carta selezionata. Se l'utente non ha mosse valide in mano, la funzione `GiocatoreHaMosse` restituisce false e il motore grafico applica automaticamente un bagliore dorato (GOLD) attorno al mazzo di pesca per suggerire visivamente l'unica azione disponibile.
- **Testo Pixel Art in Grassetto:** `DisegnaTestoPixelGrassetto` simula un font retro applicando un algoritmo ad alto contrasto che renderizza la stringa di testo in colore nero lungo molteplici direzioni (creando un'ombreggiatura spessa di contorno) prima di sovrapporvi il testo finale nel colore scelto.
- **Interpolazione e Centratura Popup:** Le immagini di fine partita (Vittoria/Sconfitta) non appaiono più in modo statico. Sfruttando la variabile `animY` moltiplicata per il frame time (`animY -= 450.0f * GetFrameTime()`), il pop-up scivola in modo fluido da fuori schermo fino al centro. Allo stesso modo, la foca della scelta colore è stata centrata matematicamente a coordinate fisse dedicate (`475.0f, 75.0f`) sopra il rispettivo testo.

3.5 ARCHIVIO.C

- In ARCHIVIO.c

Il modulo `archivio.c` è il componente software incaricato di gestire la persistenza e la visualizzazione dello storico delle partite e delle statistiche globali dei giocatori registrati. Il modulo si interfaccia direttamente con il file binario dedicato (`statistiche.dat` o `classifiche.dat`) e implementa un'interfaccia grafica (GUI) interattiva basata su Raylib, caratterizzata da un sistema di navigazione a pagine (impaginazione) e controlli di sicurezza per evitare letture fallite o overflow di memoria.

3.5.1 Architettura dei Dati e Logica di Impaginazione

Per garantire una navigazione fluida ed evitare che lo schermo venga sovraccaricato di informazioni in presenza di numerosi record, il modulo implementa un algoritmo di **impaginazione dinamica**.

La struttura dati di controllo (`ArchivioGrafica`) mantiene in memoria l'indice della pagina corrente e calcola il numero totale di pagine necessarie all'avvio. La logica matematica applicata per determinare l'intervallo di record da mostrare a schermo è basata su una costante di modulo (es. `RECORD_PER_PAGINA = 5` o `10`).

L'algoritmo calcola l'indice del primo record della pagina corrente tramite la formula:

$$\text{Offset} = \text{pagina_corrente} \times \text{RECORD_PER_PAGINA}$$

In fase di rendering, il puntatore del file o l'array dei record caricati viene scorso a partire da questo *offset*, bloccando il ciclo di disegno non appena viene raggiunta la fine della pagina o del file, ottimizzando così l'uso della CPU.

3.5.2 Flusso di Lettura del File Binario e Gestione Memoria

La lettura dei dati avviene in modo sequenziale attraverso i descrittori di file nativi del linguaggio C. Il modulo implementa una routine protetta che

previene crash del software nel caso in cui il file delle statistiche non sia ancora stato creato (ad esempio al primo avvio assoluto del gioco).

- **Apertura Controllata:** Il file viene aperto in modalità lettura binaria ("rb"). Se il puntatore FILE* restituisce NULL, il modulo intercetta l'eccezione, azzera i contatori delle pagine e mostra a schermo un messaggio informativo (es. "*Nessuna partita registrata*"), evitando tentativi di lettura su puntatori nulli.
- **Caricamento e Buffer:** I record di tipo StatisticheUtente vengono letti a blocchi tramite la funzione fread(). Per motivi di efficienza, il modulo calcola il numero totale di record presenti dividendo la dimensione totale del file per la dimensione della singola struttura dati, determinando così dinamicamente il valore di totale_pagine.

3.5.3 Rendering dell'Interfaccia e Sottosistema di Navigazione

Il rendering della schermata sfrutta la pipeline grafica di Raylib attraverso l'uso della funzione DrawTexturePro, che adatta lo sfondo dell'archivio alla risoluzione nativa di 1200×800 pixel.

Il sottosistema di navigazione è governato da tre pulsanti interattivi (btn_avanti, btn_indietro, btn_ritorna), i quali sono mappati nello spazio tramite rispettive strutture Rectangle che ne definiscono la *hitbox* reale di click. Il modulo implementa un controllo logico stringente sugli stati dei pulsanti:

1. **Pulsante Indietro:** Viene renderizzato e abilitato al click solo se $\text{pagina_corrente} > 0$.
2. **Pulsante Avanti:** Viene renderizzato e abilitato solo se $\text{pagina_corrente} < (\text{totale_pagine} - 1)$.

Questo approccio impedisce all'utente di sfiorare i limiti dell'array o del file (Underflow/Overflow di pagina), disattivando graficamente i controlli non disponibili. Al passaggio del mouse sopra le hitbox attive, viene applicato un filtro di colore (LIGHTGRAY) per fornire un feedback visivo

immediato di *hover*, mentre il click del mouse scatena la riproduzione dell'effetto sonoro d'interfaccia (PlaySound) sincronizzato con il modulo audio.c.

3.5.4 Sicurezza, Chiusura delle Risorse e Prevenzione Leak

In conformità con i requisiti di manutenzione ordinaria e gestione della memoria dinamica della GPU e della RAM di sistema, il modulo `archivio.c` espone una funzione di deallocazione (es. `ScaricaArchivio` o `DeinizializzaArchivio`).

All'uscita dalla schermata dello storico, questa routine esegue in sicurezza la funzione `UnloadTexture()` per lo sfondo e per tutti i vettori grafici dei pulsanti caricati in VRAM. Viene inoltre garantito che ogni descrittore di file aperto durante le sessioni di aggiornamento delle statistiche venga chiuso tempestivamente tramite `fclose()`, azzerando il rischio di *resource leak* o corruzione del file su disco rigido.

3.6 AUDIO.C

- In AUDIO.c

Il modulo AUDIO.C incapsula le chiamate al sottosistema audio hardware della macchina tramite le librerie Raylib, orchestrando sia i flussi musicali continui sia gli effetti sonori posizionali (feedback) di gioco.

3.6.1 Coerenza dello Streaming e Ottimizzazione del Ciclo

Nelle prime versioni del software, il brano principale musica.mp3 veniva caricato e avviato in modo ridondante all'avvio di InitAudio. Nella versione corrente, la logica è stata isolata:

- InitAudio(): Si limita ad attivare il dispositivo hardware e ad allocare in memoria RAM i file e le tracce d'ambiente, impostando i flag booleani di attivazione su true.
- PlayBackgroundMusic(): Viene invocata esclusivamente all'effettivo avvio del match dal menu di selezione. Questo evita la sovrapposizione e l'esecuzione asincrona della musica di gioco durante le delicate schermate di login, registrazione o configurazione dei form di input.
- UpdateAudio(): Inserita nel loop principale dell'applicazione, esegue l'aggiornamento costante frame per frame dei buffer di streaming musicali per assicurarne la fluidità ed evitarne l'interruzione o il clipping.

3.6.2 Sottosistema degli Effetti e Feedback Acustici

Il modulo espone funzioni condizionali collegate ai flag globali di sistema per gestire le impostazioni di attivazione scelte dall'utente:

- PlayCardSound(): Riproduce istantaneamente il file sonoro del clic o dello scarto quando viene posizionata una carta sul tavolo o premuto un pulsante della GUI.

- `PlayWinSound()` e `PlayDefeatSound()`: Intercettano lo stato finale della partita valutato da `AggiornaPartita` e riproducono immediatamente i rispettivi campioni celebrativi o di sconfitta contro l'IA.
- `ImpostaStatoMusica(bool)` e `ImpostaStatoEffetti(bool)`: Aggiornano dinamicamente i flag globali. Se la musica viene impostata su false, la funzione invoca immediatamente `StopBackgroundMusic()` per interrompere bruscamente i buffer hardware di streaming.

3.6.3 Deallocazione e Chiusura Pulita

- `CloseAudio()`: Chiamata alla chiusura dell'applicazione o all'intercettazione dell'evento `WindowShouldClose()`. Esegue le operazioni terminali di pulizia scaricando definitivamente i descrittori delle musiche e degli effetti dalla RAM e spegnendo in sicurezza la scheda audio per prevenire leak di memoria a livello di sistema operativo.

4) Testing

4.0 Bug Fixing

Nella prima parte del progetto dedicata alla modalità 1v1 in cui abbiamo affrontato più problematiche , ecco un riassunto delle risoluzioni effettuate.

1. Migliorie

Funzione e componente	Problema riscontrato (1° versione)	Come è stato risolto (2° versione)
Gestione Mazzo (Pesca e rimescolamento)	Quando il mazzo si esauriva, la funzione azzerava l'indice e rimescolava l'array originale di 108 carte. Non esisteva una degli scarti per tenere traccia delle carte già giocate degli scarti per tenere traccia delle carte già giocate , causando la potenziale "clonazione" in partita di carte già presenti nelle mani dei giocatori.	È stata aggiunta la struttura scarto per memorizzare le carte giocate. È stata creata la nuova funzione RigeneraMazzo che, quando il mazzo principale è vuoto, trasferisce gli scarti nel mazzo di pesca e li mescola. La funzione Pesca ora chiama correttamente questa logica.

Funzione e componente	Problema riscontrato (1° versione)	Come è stato risolto (2° versione)
Pescare una carta (GestisceInput)	Cliccando sul mazzo per pescare, il codice assegnava la carta e forzava sempre e immediatamente la fine del turno (p->turno = 1;). Questo violava le regole di UNO, in quanto il giocatore non poteva giocare la carta appena pescata anche se questa era compatibile.	La logica è stata aggiornata per verificare in tempo reale la giocabilità della carta appena pescata. Se la carta non può essere giocata, il turno passa; se invece è compatibile, viene attivato il flag p->puo_passare = true che permette al giocatore di effettuare un'ulteriore mossa.
Setup Partita (InizializzaPartita)	Il gioco rivelava la prima carta dal mazzo per metterla in tavola, limitandosi a scartare le carte nere finché non usciva un colore. Ignorava però completamente gli effetti speciali (es. +2, Salta, Inverti) che, da regolamento, devono attivarsi anche se estratti come prima carta.	È stato introdotto un blocco dedicato alla "Regola UNO: Effetto prima carta". Ora il gioco analizza p->cima.tipo all'avvio: se la carta è "SALTA" o "INVERTI", il turno passa subito al bot ; se è un "+2", viene applicata la penalità di due carte al giocatore primo di mano.
Meccanica "UNO" (AggiornaPartita)	Il gioco terminava brutalmente non appena il numero di carte di un giocatore arrivava a zero, senza alcuna gestione per l'iconica "chiamata" dell'ultima carta.	Sono stati aggiunti i flag ha_notificato_ultima e anima_deltaplano. Un nuovo trigger controlla se le carte in mano arrivano a 1: in tal caso, il gioco si congela per avviare un'animazione grafica ("deltaplano") e notifica visivamente l'evento.

Funzione e componente	Problema riscontrato (1° versione)	Come è stato risolto (2° versione)
IA Scelta Colore (ScegliColoreMiglioreBot)	Il BOT utilizzava una funzione dedicata per calcolare matematicamente quale colore avesse in maggioranza nella propria mano prima di giocare una carta Nera. Questo rendeva l'avversario prevedibile e appesantiva inutilmente la logica.	La funzione di calcolo è stata completamente eliminata. Ora, quando il BOT gioca un Cambia Colore o un +4, affida la scelta a una funzione nativa pseudo-casuale <code>GetRandomValue(0, 3)</code> , rendendo l'esito più imprevedibile e alleggerendo il codice.
Esperienza Utente / Aiuti Visivi (GiocatoreHaMosse e DisegnaPartita)	L'interfaccia utente era statica e non offriva alcun feedback se il giocatore non aveva carte compatibili in mano, costringendolo a dover dedurre autonomamente il blocco e cliccare alla cieca sul mazzo.	È stata creata la funzione <code>GiocatoreHaMosse</code> che ispeziona l'array della mano per cercare mosse valide. Se la funzione restituisce false e tocca al giocatore, il rendering grafico disegna un "bagliore" dorato (<code>EvidenziaCarta</code> con colore GOLD) attorno al mazzo di pesca per suggerire l'azione corretta.

Funzione e componente	Problema riscontrato (1° versione)	Come è stato risolto (2° versione)
Schermata Finale (DisegnaPartita)	Alla fine della partita, i popup di Vittoria o Sconfitta apparivano in modo istantaneo e statico al centro dello schermo (coordinate fisse), rendendo il passaggio dalla partita alla schermata finale molto brusco.	Le immagini conclusive ora sono animate. Sfruttando la variabile animY, il popup inizia da un'altezza nascosta fuori dallo schermo (800.0f) e scivola fluidamente verso l'alto fino a raggiungere l'altezza corretta moltiplicando per il frame time (animY -= 450.0f * GetFrameTime()).
Organizzazione Asset (InizializzaGrafica)	Le texture visive e i suoni erano stipati in percorsi poco strutturati e mischiati tra loro. Ad esempio, le interfacce utente condividevano la stessa cartella delle carte da gioco ("Carte/foca_popup.png").	I percorsi sono stati refattorizzati seguendo le <i>best practice</i> del game design, suddividendo logicamente i file. Ora ci sono cartelle distinte come "Sfondo/" per gli ambienti e "Animazioni/" per i popup a comparsa.

Funzione e componente	Problema riscontrato (1° versione)	Come è stato risolto (2° versione)
Risoluzione animazione foca per metterla centrale	vecchia funzione in graphics.c: <pre>if (p > anima_foca) DrawTextureEx(gfx->foca, (Vector2){30, 100}, 0, 0.45f, WHITE);</pre>	<pre>if (p->anima_foca){ // Calcoliamo le dimensioni scalate dell'immagine (0.45f è la scala attuale) float focaWidth = gfx->foca.width * 0.45f; float focaHeight = gfx->foca.height * 0.45f; // Posizioniamo l'immagine centralmente sopra la scritta Vector2 posFoca = { (float)SCHERMO_LARGHEZZA / 2.0f - focaWidth / 2.0f, (float)SCHERMO_ALTEZZA / 2.0f - focaHeight - 120.0f // Regola il -120 per alzarla o abbassarla }; DrawTextureEx(gfx->foca, posFoca, 0, 0.45f, WHITE); }</pre>

2. Migliorie aggiornate, versione 3:

Componente / Ambito	Nel Codice Vecchio (2° versione)	Nel Codice Aggiornato (3° versione)	Impatto e Beneficio Tecnico
Flusso di Rendering principale	Le funzioni BeginDrawing() e EndDrawing() erano cablate dentro DisegnaPartita.	BeginDrawing() e EndDrawing() sono state spostate nel loop del main.	Modularità elevata: Consente al main di decidere liberamente quale schermata disegnare (Menu, Auth, Gioco, Debug) all'interno dello stesso frame.
Gestione della Musica di Sottofondo	Il brano musica.mp3 veniva avviato automaticamente all'avvio in InitAudio.	Lo streaming musicale viene avviato solo all'effettivo avvio del gioco tramite PlayBackgroundMusic.	Coerenza audio: Evita che la musica di gioco parta in sottofondo durante le schermate di login, di registrazione o nel menu principale.
Logica di chiamata dell'Input	Nel main veniva eseguita una doppia chiamata ridondante a GestisciInput nello stesso frame.	La funzione GestisciInput viene invocata una sola volta, condizionata allo stato STATO_GIOCO.	Risoluzione Bug: Ottimizza le performance del ciclo di esecuzione ed evita anomalie nella lettura degli input da tastiera/ mouse.

Componente / Ambito	Nel Codice Vecchio (2° versione)	Nel Codice Aggiornato (3° versione)	Impatto e Beneficio Tecnico
Posizionamento Mascotte (Foca)	La foca per la scelta del colore appariva a coordinate fisse decentrate (30, 100).	La foca ha ora coordinate dedicate (475.0f, 75.0f) posizionandosi stabilmente sopra il testo di scelta colore.	Miglioramento UI: Il layout visivo della finestra di pop-up per la selezione del colore risulta centrato e bilanciato sullo schermo.
Pulizia della Memoria	Deallocava solo le risorse grafiche della partita e l'audio di gioco.	Integra funzioni simmetriche come ScaricaMenu per liberare texture e suoni del menu.	Prevenzione Memory Leak: Garantisce che tutta la memoria allocata per pulsanti e sfondi dei menu venga liberata alla chiusura del software.

2. Migliorie aggiornate, versione 4:

In GAME.C abbiamo modificato parti strutturali molto importanti rispetto alla versione precedente, incentrate soprattutto sulla stabilità della memoria e sulla gestione dei turni.

#1. Gestione del mazzo dinamica

Nel codice precedente, la dimensione del mazzo era data per scontata con una costante fissa (MAZZO_TOTALE).

Se per qualunque motivo il calcolo errava, si rischiava un buffer overflow. È stata introdotta la funzione **CreaMazzoCompleto(Carta *mazzo)** e il campo **p->carte_mazzo** dentro la struttura **Partita**.

In **Pesca()**, il controllo non è più solo sull'indice ma sulla consistenza del mazzo:

```
if (p->carte_mazzo <= 0 || p->indice_mazzo >= p->carte_mazzo)
RigeneraMazzo(p);
```

Questo evita che i bot o l'utente tentino di pescare dal nulla quando finiscono le carte, prevenendo i crash silenziosi all'avvio o durante le partite lunghe.

#2. Riscrittura totale della regola della pescata (bug delle regole di uno)

Nella versione precedente, se pescavi una carta e questa non era giocabile, il gioco si bloccava in uno stato intermedio (**p->puo_passare = true**) richiedendo un secondo clic manuale per passare, creando confusione.

La nuova logica in **GestisceInput**:

```
else if (!p->ha_pescato && p->giocatori[0].num_carte < MANO_MAX)
{
    Carta pescata = Pesca(p);
    p->giocatori[0].mano[p->giocatori[0].num_carte++] = pescata;
    p->ha_pescato = true;
    if (PuoGiocare(pescata, p)) {
        p->puo_passare = true; // Puoi decidere se giocarla o
passare
    } else {
        // SE NON È GIOCABILE, IL TURNO PASSA AUTOMATICAMENTE!
        p->turno = (p->turno + p->direzione + p->num_giocatori)
% p->num_giocatori;
        p->ha_pescato = false;
        p->puo_passare = false;
    }
    PlayCardSound();
}
```

Adesso segue le regole ufficiali e fluide di UNO. Quando clicchi sul mazzo se la carta è buona decidi tu se metterla giù, altrimenti il turno passa istantaneamente al BOT successivo senza freeze o input fantasma.

#3. Protezione anti-crash nel turno dei BOT

Nel vecchio codice, quando un BOT doveva pescare perché non aveva mosse, pescava alla cieca. Se il BOT aveva già troppe carte, rischiava di sfondare l'array della mano.

In fondo a **AggiornaPartita**:

```
else if (p->giocatori[b_idx].num_carte < MANO_MAX) { // <--  
Controllo di sicurezza aggiunto  
    p->giocatori[b_idx].mano[p->giocatori[b_idx].num_carte++] =  
Pesca(p);  
    p->turno = (p->turno + p->direzione + p->num_giocatori) % p->num_giocatori;  
} else {  
    // Se ha la mano piena e non può giocare, passa e basta  
    p->turno = (p->turno + p->direzione + p->num_giocatori) % p->num_giocatori;  
}
```

Anche se la partita si prolunga, nessun BOT supererà mai il limite **MANO_MAX** corrompendo i dati di gioco.

#4. Il Conflitto dei Salvataggi

Nella funzione **SalvaPartita(Partita *p)** a metà file su **salvataggi_partite.dat** si incapsula la partita dentro un **RecordSalvataggio**.

Il nuovo **ARCHIVIO.C** cerca invece il file **salvataggi.dat** leggendo direttamente **Partita**.

Per sincronizzarli, per evitare conflitti quando implementi o si usa il menu di caricamento, abbiamo fatto in modo che la funzione **SalvaPartita** dentro GAME.C usi la stessa logica di ARCHIVIO.C .

Ripulire i vecchi file `.dat` residui nella cartella fa in modo che i nuovi flussi scrivano e leggano lo stesso formato dall'inizio.

Andando oltre la semplice sintassi, emergono alcune problematiche logiche e architetturali che il compilatore non può rilevare autonomamente. Di seguito viene presentata un'analisi dettagliata dei 5 bug critici individuati e le relative soluzioni.

1.

- **File d'origine:** ARCHIVIO.c
- **Funzione:** DisegnaArchivioStorico
- **Il Problema:** All'interno di DisegnaArchivioStorico viene invocata la funzione CaricaPartiteSospeseUtente per leggere l'elenco dei match dal file binario salvataggi_dat. Poiché la funzione di disegno è inserita nel loop di rendering principale in MAIN.c (che esegue a 60 FPS tramite SetTargetFPS(60)), l'applicazione esegue l'apertura del file, l'allocazione dinamica tramite malloc, la lettura dei record, la deallocazione con free e la chiusura del file **60 volte al secondo**.
- **La Conseguenza:** Un enorme spreco di cicli, surriscaldamento del disco e evidenti micro-scatti nell'interfaccia utente.
- **La Soluzione:** Implementare una strategia di **Pre-caricamento (Caching)**. I dati dal file binario devono essere letti *una sola volta*, nell'esatto momento in cui lo stato del gioco passa da STATO_MENU_SCELTA a STATO_ARCHIVIO. La funzione di disegno si limiterà poi a ciclare sull'array già allocato stabilmente in memoria.

-Modifiche in ARCHIVIO.h

Aggiungere il prototipo della funzione di pre-caricamento:

```
// Precarica i salvataggi in memoria per evitare letture continue dal disco  
void PreparaArchivioStorico(const char *username);
```

-Modifiche in ARCHIVIO.c

Dichiarare le variabili statiche del modulo in cima al file (sotto static Texture2D btnRiprendi;), inserire la nuova funzione di preparazione e sostituire interamente la routine di disegno:

```
static Partita partiteCaricate[MAX_SALVATAGGI_PER_UTENTE];
static int nPartiteCaricate = 0;

void PreparaArchivioStorico(const char *username) {
    nPartiteCaricate = CaricaPartiteSospeseUtente(username,
partiteCaricate);
}

void DisegnaArchivioStorico(const char *username, Vector2
mousePos, int *partitaSelezionataIndice) {
    DrawTexturePro(sfondoStorico,
                    (Rectangle){ 0, 0,
(float)sfondoStorico.width, (float)sfondoStorico.height },
                    (Rectangle){ 0, 0, 1200, 800 },
                    (Vector2){ 0, 0 }, 0.0f, WHITE);

    // Utilizzo della cache in memoria RAM invece dell'I/O su
disco
    if (nPartiteCaricate == 0) {
        int larghezzaTestoVuoto = MeasureText("NESSUN RECORD IN
ARCHIVIO", 24);
        DrawText("NESSUN RECORD IN ARCHIVIO", (1200 -
larghezzaTestoVuoto) / 2, 400, 24, LIGHTGRAY);
        return;
    }

    int centroColonnaData = 260;
    int centroColonnaEvento = 560;
    int centroColonnaPulsante = 890;
    int rigaInizialeY = 320;
    int scostamentoY = 78;

    for (int i = 0; i < nPartiteCaricate; i++) {
        int centroRigaY = rigaInizialeY + (i * scostamentoY);
        char dataSolo[12] = "";
        char oraSolo[8] = "";

        if (strlen(partiteCaricate[i].data_salvataggio) >= 16) {
```

```

        strncpy(dataSolo,
partiteCaricate[i].data_salvataggio, 10);
        dataSolo[10] = '\0';
        strncpy(oraSolo, partiteCaricate[i].data_salvataggio
+ 11, 5);
        oraSolo[5] = '\0';
    } else {
        strcpy(dataSolo, "21/05/2026");
        strcpy(oraSolo, "19:00");
    }

    int larghezzaData = MeasureText(dataSolo, 20);
    int larghezzaOra = MeasureText(oraSolo, 20);

    DrawText(dataSolo, centroColonnaData - (larghezzaData /
2), centroRigaY - 14, 20, WHITE);
    DrawText(oraSolo, centroColonnaData - (larghezzaOra /
2), centroRigaY + 8, 20, WHITE);

    char stringaEvento[32];
    if (partiteCaricate[i].num_giocatori == 2) {
        strcpy(stringaEvento, "1 VS 1 (BOT)");
    } else {
        strcpy(stringaEvento, "MULTIPLAYER (1vs3)");
    }

    int larghezzaEvento = MeasureText(stringaEvento, 20);
    DrawText(stringaEvento, centroColonnaEvento -
(larghezzaEvento / 2), centroRigaY - 10, 20, WHITE);

    float larghPulsante = 140.0f;
    float altPulsante = 42.0f;
    Rectangle rectPulsante = {
        (float)(centroColonnaPulsante - (larghPulsante /
2)),
        (float)(centroRigaY - (altPulsante / 2)),
        larghPulsante,
        altPulsante
    };

    bool collisioneMouse = CheckCollisionPointRec(mousePos,
rectPulsante);
    Rectangle srcRec = { 0.0f, 0.0f,
(float)btnRiprendi.width, (float)btnRiprendi.height };

    if (collisioneMouse) {

```

```

        DrawTexturePro(btnRiprendi, srcRec, rectPulsante,
(Vector2){ 0, 0 }, 0.0f, LIGHTGRAY);
        if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT)) {
            PlayCardSound();
            *partitaSelezionataIndice = i;
        }
    } else {
        DrawTexturePro(btnRiprendi, srcRec, rectPulsante,
(Vector2){ 0, 0 }, 0.0f, WHITE);
    }
}
}

```

-Modifiche in MAIN.c

Innescare il pre-caricamento aggiornando l'intercettazione dell'azione VAI_ARCHIVIO nello switch del menu di scelta:

```

} else if (azione == VAI_ARCHIVIO && utente_loggato) {
    OttieniStatisticheUtente(partita.utente_corrente,
&vittorieUtente, &sconfitteUtente);
    PreparaArchivioStorico(partita.utente_corrente); //
Iniezione del pre-caricamento dei dati
    statoAttuale = STATO_ARCHIVIO;
}

```

2. Pesca Infinita e Turno Bloccato (Bug di Logica di Gameplay)

- File d'origine: GAME.c
- Funzione: GestisceInput
- Il Problema: Al momento della pesca, il codice esegue il calcolo matematico del turno successivo solo se l'utente *non* può giocare la carta pescata. Se la carta è valida, il turno rimane impostato a 0 (turno del giocatore umano) e il flag p->puo_passare diventa true. Tuttavia, non essendoci un pulsante a schermo per passare manualmente il turno, il giocatore si ritrova bloccato: può continuare a cliccare sul mazzo per pescare all'infinito o essere forzato a giocare contro la propria strategia.
- La Conseguenza: Violazione delle regole ufficiali del gioco UNO e potenziale soft-lock dell'esperienza di gioco.

- La Soluzione: Introdurre un flag di controllo ha_pescato. Il primo clic permette la pescata singola. Se la carta non è giocabile, il sistema sblocca un secondo clic sul mazzo (o su un apposito pulsante) che funge da comando "Passa il Turno", resettando correttamente i flag per l'avversario.

-Modifiche in GAME.h

All'interno della struttura Partita, dichiarare la nuova variabile di stato:

```
bool ha_pescato; // Traccia se l'utente ha già effettuato la
pescata di turno
```

-Modifiche in GAME.c

Sostituzione integrale della funzione GestisceInput:

```
void GestisceInput(Partita *p) {
    if (p->partita_finita || p->timer_messaggio > 0 || p-
>anima_deltaplano) return;

    if (IsKeyPressed(KEY_ESCAPE)) {
        PlayCardSound();
        p->richiedi_uscita = true;
        return;
    }

    Vector2 m = GetMousePosition();
    if (!p->is_guest) {
        Rectangle btnSospendi = { 990, 700, 180, 70 };
        if (CheckCollisionPointRec(m, btnSospendi) &&
IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
            PlayCardSound();
            p->richiedi_uscita = true;
            return;
        }
    }

    if (p->scegli_colore) {
        for (int i = 0; i < 4; i++) {
            Rectangle r = { SCHERMO_LARGHEZZA/2.0f - 160 + i*85,
SCHERMO_ALTEZZA/2.0f - 40, 75, 75 };

```

```

        if (CheckCollisionPointRec(m, r) &&
IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
            p->colore_attuale = (Colore)i;
            p->scegli_colore = false;
            p->anima_foca = false;
            p->turno = p->prossimo_turno;
        }
    }
    return;
}

if (p->turno != 0 || p->animando) return;

// Gestione clic sul mazzo di pesca
if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON) &&
    CheckCollisionPointRec(m, (Rectangle)
{ SCHERMO_LARGHEZZA/2.0f - 160, SCHERMO_ALTEZZA/2.0f - 75, 100,
150 }))) {

    if (p->puo_passare) {
        // Secondo clic: il giocatore decide di passare il
turno volontariamente
        p->turno = (p->turno + p->direzione + p-
>num_giocatori) % p->num_giocatori;
        p->puo_passare = false;
        p->ha_pescato = false;
        PlayCardSound();
    }
    else if (!p->ha_pescato && p->giocatori[0].num_carte <
MANO_MAX) {
        // Primo clic: effettua la pescata di regola
        Carta pescata = Pesca(p);
        p->giocatori[0].mano[p->giocatori[0].num_carte++] =
pescata;
        p->ha_pescato = true;

        if (!PuoGiocare(pescata, p)) {
            p->puo_passare = true; // Abilita il secondo
clic condizionale per passare
        }
        PlayCardSound();
    }
}

int spacing = 65;
int totalW = (p->giocatori[0].num_carte - 1) * spacing;

```

```

int startX = SCHERMO_LARGHEZZA/2 - totalW / 2;

for (int i = p->giocatori[0].num_carte - 1; i >= 0; i--) {
    Rectangle r = {startX + i*spacing - 50, SCHERMO_ALTEZZA
- 175, 100, 150};
    if (CheckCollisionPointRec(m, r) &&
IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
        if (PuoGiocare(p->giocatori[0].mano[i], p)) {
            p->carta_animata = p->giocatori[0].mano[i];
            for (int k = i; k < p->giocatori[0].num_carte -
1; k++) {
                p->giocatori[0].mano[k] = p-
>giocatori[0].mano[k+1];
            }
            p->giocatori[0].num_carte--;
            p->animando = true;
            p->timer_anim = 0;
            p->durata_animazione = 0.3f;
            p->da_giocatore = 0;
            p->inizio_animazione = (Vector2){(float)startX +
i*spacing, (float)SCHERMO_ALTEZZA - 100};
            p->fine_animazione = (Vector2)
{SCHERMO_LARGHEZZA/2.0f + 110.0f, SCHERMO_ALTEZZA/2.0f};

            // Reset degli stati a seguito di una carta
giocata con successo
            p->ha_pescato = false;
            p->puo_passare = false;
        }
        break;
    }
}
}
}

```

3. & 4. Accesso Admin Istantaneo e Clean Code

- **File d'origine:** MAIN.c & MENU.c

1. UX Sgradevole: Il controllo delle credenziali dell'amministratore avviene in tempo reale all'interno del loop grafico del main. Non appena l'utente digita l'ultimo carattere della password ("1234"),

l'applicazione effettua un salto di stato immediato a STATO_DEBUG senza attendere la conferma dell'utente.

2. All'inizio di **GestisciSchermataAuth** in MENU.c è presente un controllo esplicito sullo stato STATO_DEBUG. Tuttavia, questa porzione non verrà mai eseguita poiché in MAIN.c lo stato di debug viene isolato all'interno di un proprio case e non invoca in alcun modo questo modulo.

- **La Soluzione:** Rimuovere il blocco di codice non raggiungibile in MENU.c e incapsulare la validazione del profilo amministratore all'interno dell'evento di pressione del tasto KEY_ENTER.

-Sostituzione di GestisciSchermataAuth in MENU.c

```
StatoGioco GestisciSchermataAuth(MenuGrafica *mg, StatoGioco
stato) {
    Vector2 mouse = GetMousePosition();
    mg->frame_counter++;
    float ottimaleLarghezza = 312.0f;
    float ottimaleAltezza = 72.0f;
    if (mg->tex_utente.id > 0) {
        ottimaleAltezza = (float)mg->tex_utente.height *
(optimaleLarghezza / (float)mg->tex_utente.width);
    }

    float posizionaX = 615.0f;
    float posizionaNickY = 442.0f - (ottimaleAltezza / 2.0f);
    float posizionaPassY = 568.0f - (ottimaleAltezza / 2.0f);
    Rectangle rNickDest = { posizionaX, posizionaNickY,
ottimaleLarghezza, ottimaleAltezza };
    Rectangle rPassDest = { posizionaX, posizionaPassY,
ottimaleLarghezza, ottimaleAltezza };

    if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
        if (CheckCollisionPointRec(mouse, rNickDest)) mg-
>campo_attivo = 1;
        else if (CheckCollisionPointRec(mouse, rPassDest)) mg-
>campo_attivo = 2;
        else mg->campo_attivo = 0;
    }
}
```

```

GestisceInputTesto(mg->nickname, &mg->campo_attivo, 1);
GestisceInputTesto(mg->password, &mg->campo_attivo, 2);

Texture2D texSfondo = (stato == STATO_REGISTRATI) ? mg-
>sfondo_registrati : mg->sfondo_accedi;
if (texSfondo.id > 0) {
    DrawTexturePro(texSfondo, (Rectangle){0, 0,
(float)texSfondo.width, (float)texSfondo.height},
(Rectangle){0, 0, 1200, 800}, (Vector2)
{0, 0}, 0, WHITE);
}

float spessoreBordo = 5.0f;
if (mg->campo_attivo == 1 && mg->tex_utente.id > 0) {
    Rectangle rGlow = { rNickDest.x - spessoreBordo,
rNickDest.y - spessoreBordo, rNickDest.width + (spessoreBordo *
2), rNickDest.height + (spessoreBordo * 2) };
    DrawTexturePro(mg->tex_utente, (Rectangle){0, 0,
(float)mg->tex_utente.width, (float)mg->tex_utente.height},
rGlow, (Vector2){0, 0}, 0, GOLD);
} else if (mg->campo_attivo == 2 && mg->tex_password.id > 0)
{
    Rectangle rGlow = { rPassDest.x - spessoreBordo,
rPassDest.y - spessoreBordo, rPassDest.width + (spessoreBordo *
2), rPassDest.height + (spessoreBordo * 2) };
    DrawTexturePro(mg->tex_password, (Rectangle){0, 0,
(float)mg->tex_password.width, (float)mg->tex_password.height},
rGlow, (Vector2){0, 0}, 0, GOLD);
}

if (mg->tex_utente.id > 0) {
    DrawTexturePro(mg->tex_utente, (Rectangle){0, 0,
(float)mg->tex_utente.width, (float)mg->tex_utente.height},
rNickDest, (Vector2){0, 0}, 0, WHITE);
}
if (mg->tex_password.id > 0) {
    DrawTexturePro(mg->tex_password, (Rectangle){0, 0,
(float)mg->tex_password.width, (float)mg->tex_password.height},
rPassDest, (Vector2){0, 0}, 0, WHITE);
}

float testonickDim = 22.0f;
float testopassDim = 26.0f;
float allineamentoTestoY_Nick = posizionaNickY +
(ottimaleAltezza / 2.0f) - (testonickDim / 2.0f);

```

```

    float allineamentoTestoY_Pass = posizionaPassY +
(ottimaleAltezza / 2.0f) - (testopassDim / 2.0f) + 2.0f;
    float allineamentoPlaceholderY_Pass = posizionaPassY +
(ottimaleAltezza / 2.0f) - (16.0f / 2.0f);
    float inputNickX = posizionaX + 115.0f;
    float inputPassX = posizionaX + 115.0f;

    if (strlen(mg->nickname) == 0) {
        DrawText("max 10 caratteri", (int)inputNickX,
(int)allineamentoTestoY_Nick + 2, 16, WHITE);
    } else {
        DrawText(mg->nickname, (int)inputNickX,
(int)allineamentoTestoY_Nick, (int)testonickDim, BLACK);
    }

    char stars[12] = {0};
    if (strlen(mg->password) == 0) {
        DrawText("max 10 caratteri", (int)inputPassX,
(int)allineamentoPlaceholderY_Pass, 16, WHITE);
    } else {
        for (int i = 0; i < (int)strlen(mg->password); i++)
stars[i] = '*';
        DrawText(stars, (int)inputPassX,
(int)allineamentoTestoY_Pass, (int)testopassDim, BLACK);
    }

    if (mg->campo_attivo == 1) {
        if (((mg->frame_counter / 20) % 2) == 0) {
            int spostamentoTesto = MeasureText(mg->nickname,
(int)testonickDim);
            DrawText("|", (int)(inputNickX + spostamentoTesto),
(int)allineamentoTestoY_Nick, (int)testonickDim, MAROON);
        }
    } else if (mg->campo_attivo == 2) {
        if (((mg->frame_counter / 20) % 2) == 0) {
            int spostamentoTesto = MeasureText(stars,
(int)testopassDim);
            DrawText("|", (int)(inputPassX + spostamentoTesto),
(int)allineamentoTestoY_Pass - 2, (int)testopassDim, MAROON);
        }
    }

    if (codice_errore > 0 && codice_errore != 99) {
        const char* msg = (codice_errore == 1) ? "UTENTE NON
TROVATO" :

```

```

                                (codice_errore == 2) ? "PASSWORD
ERRATA" :
                                (codice_errore == 3) ? "NICKNAME GIA'
IN USO" : "SPAZIO NON CONSENTITO";
    int tw = MeasureText(msg, 30);
    DrawRectangle(600 - tw / 2 - 15, 100, tw + 30, 50,
Fade(BLACK, 0.8f));
    DrawText(msg, 600 - tw / 2, 110, 30, RED);
}

// Sincronizzazione della validazione con la pressione del
tasto INVIO
    if (IsKeyPressed(KEY_ENTER) && strlen(mg->nickname) > 0 &&
strlen(mg->password) > 0) {
        if (stato == STATO_REGISTRATI) {
            if (EsisteNickname(mg->nickname)) {
                codice_errore = 3;
            } else {
                RegistraNuovoUtenteInTabella(mg->nickname, mg-
>password);
                SalvaLogStoricoAzione(mg->nickname, mg-
>password, 0);
                PlaySound(suono_bottone);
                return STATO_MENU_SCELTA;
            }
        } else {
            int res = VerificaCredenzialiDettagliata(mg-
>nickname, mg->password);
            if (res == 99) {
                PlaySound(suono_bottone);
                codice_errore = 0;
                return STATO_DEBUG; // Restituzione coerente
dello stato amministratore
            }
            if (res == 0) {
                AggiornaTimestampAccesso(mg->nickname);
                SalvaLogStoricoAzione(mg->nickname, mg-
>password, 1);
                PlaySound(suono_bottone);
                codice_errore = 0;
                return STATO_MENU_SCELTA;
            } else {
                codice_errore = res;
                memset(mg->password, 0, 12);
            }
        }
    }
}

```

```

}

if (IsKeyPressed(KEY_ESCAPE)) {
    codice_errore = 0;
    return STATO_MENU;
}
return stato;
}

```

-Sostituzione dei blocchi case di Autenticazione in MAIN.c

```

case STATO_ACCEDI:
case STATO_REGISTRATI: {
    StatoGioco prossimo = GestisciSchermataAuth(&menuG,
statoAttuale);

    // Instradamento controllato al pannello di controllo
Amministratore
    if (prossimo == STATO_DEBUG) {
        conteggioRigheLog = 0;
        FILE *f = fopen("./registro_accessi.bin", "rb");
        if (f != NULL) {
            while (fread(&logAdminTabella[conteggioRigheLog],
sizeof(RecordAccesso), 1, f) == 1) {
                conteggioRigheLog++;
                if (conteggioRigheLog >= 40) break;
            }
            fclose(f);
        }
        strcpy(menuG.nickname, "");
        strcpy(menuG.password, "");
        statoAttuale = STATO_DEBUG;
        break;
    }
    // Gestione standard del flusso di autenticazione utente
    if (prossimo == STATO_MENU_SCELTA) {
        utente_loggato = true;
        partita.is_guest = false;
        strncpy(partita.utente_corrente, menuG.nickname, 11);
        partita.utente_corrente[11] = '\0';

        if (statoAttuale == STATO_ACCEDI) {
            ScriviLogAccessoBinario("LOGIN",
partita.utente_corrente, menuG.password);
        }
    }
}

```

```

        statoAttuale = prossimo;
    } else {
        if (statoAttuale == STATO_REGISTRATI && prossimo ==
STATO_MENU) {
            ScriviLogAccessoBinario("REGISTRAZIONE",
menuG.nickname, menuG.password);
        }
        statoAttuale = prossimo;
    }
    break;}

```

5.

- **File d'origine:** MENU.c
- **Funzione:** RegistraNuovoUtenteInTabella
- **Il Problema:** Quando si copiano stringhe di testo con dimensioni limitate tramite la funzione standard `strncpy(u.nickname, nick, 10)` , se l'input inserito dall'utente satura la lunghezza massima impostata (es. 10 caratteri), la funzione non inserisce automaticamente il carattere terminatore di stringa .
- **La Conseguenza:** Attualmente il codice evita il crash solo perché viene eseguito preventivamente un ciclo `memset` azzerando la struttura. Se in futuro il `memset` dovesse essere modificato o rimosso, la stringa rimarrebbe "sporca" in memoria provocando corruzioni di dati in lettura.
- **La Soluzione:** Forzare manualmente l'inserimento del terminatore nell'ultimo byte disponibile dell'array subito dopo l'operazione di copia.

-Sostituzione della funzione in MENU.c

```
static void RegistraNuovoUtenteInTabella(const char* nick, const
char* pass) {
    FILE *file = fopen("utenti.dat", "ab");
    if (file != NULL) {
        UtenteRegistrato u;
        memset(&u, 0, sizeof(UtenteRegistrato));

        strncpy(u.nickname, nick, 10);
        u.nickname[10] = '\\0'; // Inserimento di sicurezza del
terminatore nativo

        strncpy(u.password, pass, 10);
        u.password[10] = '\\0'; // Inserimento di sicurezza del
terminatore nativo

        time_t t = time(NULL);
        struct tm *tm_info = localtime(&t);
        if (tm_info) strftime(u.ultimo_accesso, 24, "%d/%m/%Y
%H:%M:%S", tm_info);

        u.ultima_azione = 0; // 0 = REGISTRAZIONE

        fwrite(&u, sizeof(UtenteRegistrato), 1, file);
        fclose(file);
    }
}
```

Per garantire la stabilità assoluta del software ed evitare bug nascosti in fase di runtime, all'interno del file header di definizione (es. ARCHIVIO.h o MENU.h), gli array di caratteri hanno una dimensione minima di **11 elementi** per ospitare il carattere terminatore (10 caratteri utili + 1 terminatore).

Invece di dichiarare come `char nickname[10]` che provocherà un buffer overflow, corrompendo la variabile adiacente in memoria RAM, La configurazione corretta nel file **.h** sarà:

```
typedef struct {
    char nickname[11];          // 10 caratteri + 1 per il '\\0'
```

```
char password[11];          // 10 caratteri + 1 per il '\0'
char ultimo_accesso[24];
int ultima_azione;
} UtenteRegistrato;
```

4. Inizializzazione della variabile `ha_pescato`

Avendo aggiunto il flag `bool ha_pescato` all'interno della struct principale `Partita`, è indispensabile garantirne il corretto azzeramento all'avvio. In linguaggio C, i nuovi campi non esplicitamente definiti mantengono valori casuali residui presenti in RAM (*valori spazzatura*).

- **Rischio:** Se il flag partisse per errore come true, il giocatore si ritroverebbe nell'impossibilità di pescare la sua prima carta di gioco.
- **Soluzione:** all'interno di `GAME.c` la funzione di inizializzazione della sessione viene inserito esplicitamente il reset di stato:

```
p->puo_passare = false;
p->ha_pescato = false; // Reset dello stato di pesca all'avvio
del match
```

4.1 Test

Tabella 1 di test

Codice Test	Test	Descrizione	Input	Risultato Atteso	Risultato Ottenuto
T001	Login utente valido	Login utente valido	Nickname e password corretti in STATO_ACCE DI	Accesso consentito, utente_loggato = true, chiamata a ScriviLogAccessoBinario() e transizione al menu di scelta.	Accesso riuscito e record di log scritto in registro_accessi.bin.
T002	Login utente non valido	Login utente non valido	Password errata o nickname inesistente	Accesso negato, valorizzazione del codice di errore grafico a schermo.	Credenziali errate, accesso negato.
T003	Registrazione nuovo utente	Registrazione nuovo utente	Credenziali valide e nickname non duplicato	Scrittura del record struttura UtenteRegistrato dentro utenti.dat e conferma visiva.	Registrazione completata correttamente.
T004	Accesso come Guest	Autenticazione in modalità Ospite	Click sul pulsante "Ospite" nel menu principale	is_guest = true, bypass del log binario, flag nascondi_archivio a true nel menu successivo.	Navigazione consentita come ospite con funzioni storiche disabilitate.
T005	Avvio Partita 1vs1	Inizializzazione match contro un singolo bot	Click su VAI_1VS1 in STATO_MENU_SCELTA	Esecuzione di InizializzaPartita(p, 2), avvio traccia audio e transizione a STATO_GIOCO.	Partita 1vs1 inizializzata correttamente.
T006	Avvio Partita Multigiocatore	Inizializzazione match a 4 giocatori	Click su VAI_MULTIGIOCATORE	Esecuzione di InizializzaPartita(p, 4), distribuzione delle mani e transizione a STATO_GIOCO.	Partita a 4 giocatori avviata con successo.

Codice Test	Test	Descrizione	Input	Risultato Atteso	Risultato Ottenuto
T007	Giocata carta valida	Verifica compatibilità e scarto carta	Click su una carta idonea per colore o tipo	Rimozione carta dalla mano del giocatore, attivazione di p->animando, riproduzione effetto sonoro.	Carta giocata correttamente, turno aggiornato.
T008	Pesca carta dal mazzo	Azione di pesca condizionata	Click sul mazzo di pesca coperto	Aggiunta di una carta alla struttura mano (se inferiore a MANO_MAX), avanzamento automatico del turno.	Carta aggiunta alla mano con successo.
T009	Intelligenza Artificiale e Bot	Calcolo ed esecuzione mossa dell'IA	Turno di gioco passato a un indice Bot (turno > 0)	Accumulatore di tempo IA elabora lo scarto o la pesca; rendering animazioni foca/ deltaplano.	Il Bot esegue la mossa in autonomia e passa il turno.
T010	Sospensione partita (ESC)	Salvataggio persistente di una partita attiva	Pressione del tasto ESC (con utente non guest)	Chiamata a SalvaPartitaSospesa(), arresto audio di gioco e transizione a STATO_MENU_SCELTA.	Partita congelata su file e uscita al menu completata.
T011	Ripristino da Archivio	Caricamento partita salvata dallo storico	Selezione di uno slot valido in STATO_ARCHIVIO	Ripristino della struttura Partita. Reset forzato dei flag volatili (blocco_input, animando, timer_attesa), rimozione del record usato.	Partita ripresa esattamente dallo stato salvato senza blocchi di input.
T012	Visualizzazione Statistiche	Lettura dati di rendimento dell'utente	Navigazione in STATO_STATISTICHE da Archivio	Chiamata a OttieniStatisticheUtente(), rendering centrato dei valori di vittorie (verde) e sconfitte (rosso).	Statistiche utente visualizzate correttamente.

Codice Test	Test	Descrizione	Input	Risultato Atteso	Risultato Ottenuto
T013	Configurazione Impostazioni Audio	Switch dinamico dello stato audio	Click su rectOnMusica o rectOnAudio	Inversione dei flag booleani locali e muting/unmuting immediato delle tracce.	Stato audio aggiornato in tempo reale nelle textures On/Off.
T014	Accesso Pannello Admin (Debug)	Monitoraggio dei log di sistema amministrativi	Login con user "admin" e password "1234"	Bypass delle funzioni utente standard, caricamento sequenziale di massimo 40 record da registro_accessi.bin in logAdminTabella.	Schermata di debug tabellare mostrata con successo.

4.2 Sviluppi Futuri

Per i vincoli, derivanti dalla natura monolitica del prototipo accademico, definiscono le linee guida per la futura evoluzione industriale del progetto:

1. Integrazione del Sottosistema di Networking(Multigiocatore):

L'attuale architettura vincola l'esperienza di gioco a sessioni locali tra un singolo utente e gli agenti artificiali. Uno sviluppo futuro prioritario prevede la scomposizione del ciclo di input/output per accogliere un'architettura Client-Server basata su protocolli di trasporto TCP/IP. Questo impianto consentirà di abilitare sessioni multigiocatore sincronizzate in rete.

2.Implementazione della Programmazione Concorrente (Multithreading):

L'introduzione del paradigma concorrente tramite la libreria pthread.h o i thread nativi dello standard ISO C11 permetterà di disaccoppiare il thread principale di rendering grafico (gestito da Raylib) dai thread computazionali asincroni dedicati all'elaborazione delle decisioni dell'IA o al caricamento dei flussi audio. In questo modo si elimineranno alla radice potenziali fenomeni di micro-stuttering grafico.

3. Migrazione verso Strutture DBMS e Crittografia della Persistenza:

Il sistema di persistenza dei dati si affida correntemente a file binari locali memorizzati in modo sequenziale (salvataggi_partite.dat). Per incrementare i livelli di sicurezza, scalabilità e integrità referenziale, si prospetta l'adozione di un DBMS relazionale embedded quale SQLite, supportato da algoritmi di hashing crittografico per la protezione delle credenziali d'accesso degli utenti.

Conclusione

Con la conclusione dello sviluppo di questo applicativo dedicato al gioco dell'UNO, giungiamo al termine di un percorso che ha rappresentato una vera e propria pietra miliare per il nostro percorso accademico e personale.

Per noi tre - Sara, Giordano e Samuele - questo lavoro ha assunto un significato speciale: ha segnato il nostro primissimo progetto informatico. Passare dalla teoria dei libri alla progettazione pratica, vedendo le nostre linee di codice prendere vita e trasformarsi in un gioco completo e funzionante, è stata un'emozione indescrivibile.

È stato un viaggio incredibilmente stimolante, che ci ha messi alla prova di fronte a logiche di gioco complesse e sfide tecniche sempre nuove, costringendoci a superare i nostri limiti e a trovare soluzioni creative. Al tempo stesso, è stato un percorso decisamente divertente: i momenti di confronto, le sessioni di debugging condivise e l'entusiasmo nel vedere ogni piccola funzionalità andare al suo posto hanno reso le ore di lavoro un piacere e hanno rafforzato la nostra sinergia come gruppo.

Ma, sopra ogni cosa, questa esperienza è stata profondamente formativa. Non abbiamo solo affinato le nostre competenze tecniche di programmazione e gestione del progetto, ma abbiamo compreso il reale valore del gioco di squadra, dell'ascolto reciproco e della collaborazione. Guardando il traguardo raggiunto, non possiamo che ritenerci fieri e soddisfatti del software che abbiamo creato, grati per tutto ciò che abbiamo imparato insieme e pronti, con una consapevolezza nuova, ad affrontare le future sfide che il mondo dell'informatica ci riserverà.

-Grazie